

LAPORAN PRAKTIKUM 1

Pointer 1



Disusun Oleh:

Ilyas Rahman Salim (5325600063)

Program Studi D4 Teknologi Rekayasa Multimedia

Departemen Teknologi Multimedia Kreatif

Politeknik Elektronika Negeri Surabaya

2026

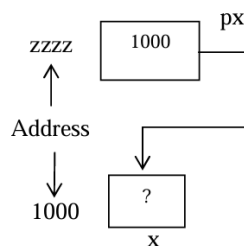
A. TUJUAN PEMBELAJARAN

1. Memahami konsep dari variabel pointer
2. Memahami cara Mengakses Isi Suatu Variabel Melalui Pointer
3. Memahami cara Mengakses dan Mengubah isi Suatu Variabel Pointer

B. DASAR TEORI

Konsep Dasar Pointer

Variabel pointer sering dikatakan sebagai variabel yang menunjuk ke obyek lain. Pada kenyataan yang sebenarnya, variabel pointer berisi alamat dari suatu obyek lain (yaitu obyek yang dikatakan ditunjuk oleh pointer). Sebagai contoh, `px` adalah variabel pointer dan `x` adalah variabel yang ditunjuk oleh `px`. Kalau `x` berada pada alamat memori (alamat awal) 1000, maka `px` akan berisi 1000. Sebagaimana diilustrasikan pada Gambar 24.1 di bawah ini



Gambar 24.1 Variabel pointer `px` menunjuk ke variabel `x`

Mendeklarasikan Variabel Pointer

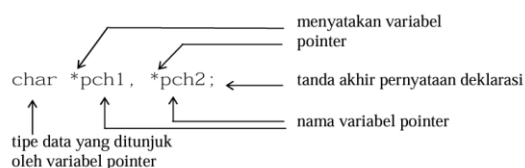
Suatu variabel pointer dideklarasikan dengan bentuk sebagai berikut :

tipe *nama_variabel

dengan tipe dapat berupa sembarang tipe yang sudah dibahas pada bab-bab sebelumnya, maupun bab-bab berikutnya. Adapun `nama_variabel` adalah nama dari variabel pointer. Sebagai contoh :

```
int *px;           //contoh 1
char *pch1, *pch2; //contoh 2
```

Contoh pertama menyatakan bahwa `px` adalah variabel pointer yang menunjuk ke suatu data bertipe `int`, sedangkan contoh kedua masing `pch1` dan `pch2` adalah variabel pointer yang menunjuk ke data bertipe `char`.



Gambar 19.2 Ilustrasi pendeklarasian variabel pointer

Mengatur Pointer agar Menunjuk ke Variabel Lain

Agar suatu pointer menunjuk ke variabel lain, mula-mula pointer harus diisi dengan alamat dari variabel yang akan ditunjuk. Untuk menyatakan alamat dari suatu variabel, operator `&` (operator alamat, bersifat unary) bisa dipergunakan, dengan menempatkannya di depan nama variabel. Sebagai contoh, bila `x` dideklarasikan sebagai variabel bertipe `int`, maka

`&x`

berarti “alamat dari variabel x”. Adapun contoh pemberian alamat x ke suatu variabel pointer px (yang dideklarasikan sebagai pointer yang menunjuk ke data bertipe int) yaitu :

```
px = &x;
```

Pernyataan di atas berarti bahwa px diberi nilai berupa alamat dari variabel x. Setelah pernyataan tersebut dieksekusi barulah dikatakan bahwa px menunjuk ke variabel x.

Mengakses Isi Suatu Variabel Melalui Pointer

Jika suatu variabel sudah ditunjuk oleh pointer, variabel yang ditunjuk oleh pointer tersebut dapat diakses melalui variabel itu sendiri (pengaksesan langsung) ataupun melalui pointer (pengaksesan tak langsung). Pengaksesan tak langsung dilakukan dengan menggunakan operator indirection (tak langsung) berupa simbol * (bersifat unary). Contoh penerapan operator * yaitu :

```
*px
```

yang menyatakan “nilai/value yang ada pada alamat/address px” . Sebagai contoh jika y bertipe int, maka sesudah dua pernyataan berikut

```
px = &x;
```

```
y = *px;
```

y akan berisi nilai yang sama dengan nilai x.

Kedua pernyataan di atas

```
px = &x;
```

```
y = *px;
```

sebenarnya dapat digantikan dengan sebuah pernyataan berupa

```
y = x;
```

Seandainya pada program di atas tidak terdapat pernyataan

```
px = &x;
```

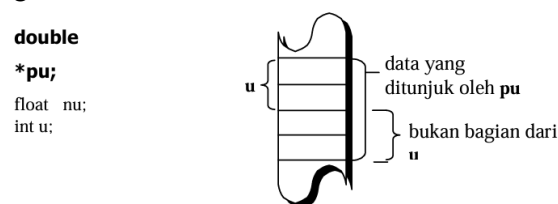
namun terdapat pernyataan

```
y = *px;
```

maka y tidaklah berisi nilai x, sebab px belum diatur agar menunjuk ke variabel x. Hal seperti ini harap diperhatikan. Kalau program melibatkan pointer, dan pointer belum diinisialisasi, ada kemungkinan akan terjadi masalah yang dinamakan “bug” yang bisa mengakibatkan komputer tidak dapat dikendalikan (hang).

Selain itu tipe variabel pointer dan tipe data yang ditunjuk harus sejenis. Bila tidak sejenis maka akan terjadi hasil yang tidak diinginkan.

Untuk lebih jelasnya lihat gambar berikut.



Gambar 19.3 Ilustrasi kesalahan yang terjadi karena tipe tidak

Mengakses dan Mengubah isi Suatu Variabel Pointer

Contoh berikut memberikan gambaran tentang pengubahan isi suatu variabel secara tak langsung (yaitu melalui pointer). Mula-mula pd dideklarasikan sebagai pointer yang menunjuk ke suatu data bertipe float dan d sebagai variabel bertipe float. Selanjutnya

```
d = 54.5;
```

digunakan untuk mengisi nilai 54,5 secara langsung ke variabel d. Adapun

```
pd = &d;
```

digunakan untuk memberikan alamat dari d ke pd. Dengan demikian pd menunjuk ke variabel d. Sedangkan pernyataan berikutnya

```
*pd = *pd + 10;      (atau: *pd += 10; )
```

merupakan instruksi untuk mengubah nilai variabel d secara tak langsung. Perintah di atas berarti “jumlahkan yang ditunjuk pd dengan 10 kemudian berikan ke yang ditunjuk oleh pd”, atau identik dengan pernyataan

```
d = d + 10;
```

Akan tetapi, seandainya tidak ada instruksi

```
pd = &d;
```

maka pernyataan

```
*pd = *pd + 10;
```

tidaklah sama dengan

```
d = d + 10;
```

C. PERCOBAAN

Untuk setiap program di bawah ini,

- gambarkan ilustrasi alokasi memori dari setiap baris pernyataan yang diproses
- perkirakan hasil eksekusinya

```
1. main(){
    int y, x = 87;
    int *px;

    px = &x;
    y = *px;

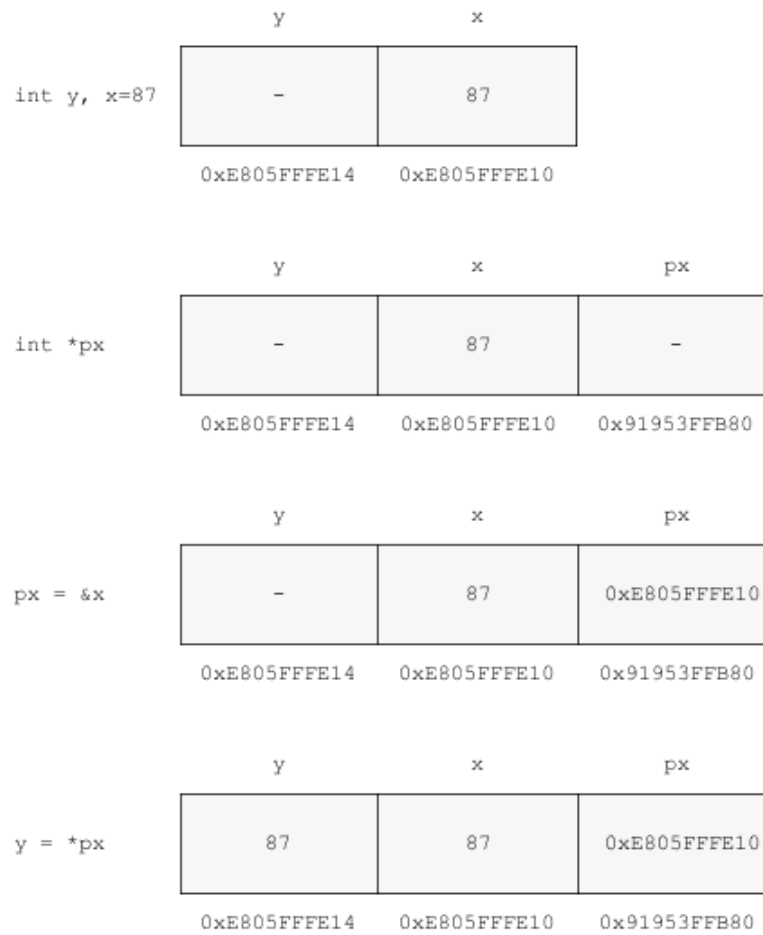
    printf("Alamat x   = %p\n", &x);
    printf("Isi px     = %p\n", px);
    printf("Isi x      = %d\n", x);
    printf("Nilai yang ditunjuk oleh px = %d\n", *px);
    printf("Nilai y    = %d\n", y);
}
```

Jawab :

a. Ilustrasi alokasi memori

Alur :

1. Inisialisasi int y dan x berisi 87
2. Inisialisasi sebuah pointer *px
3. Pointer px mengarahkan alamat ke int x
4. Int y merujuk pada *px



Ilustrasi alokasi memori

- b. Perkiraan hasil eksekusi
Berikut adalah hasil eksekusi pointer

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2> cd 'c:\Users\Ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\output'
PS C:\Users\Ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\output> & .\pointer 1 soal 1.exe
Alamat x = 000000C9183FFC60
Isi x = 87
Nilai yang ditunjuk oleh px = 87
Nilai y = 87
PS C:\Users\Ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\output>

```

Gambar hasil eksekusi pointer

Berdasarkan gambar tersebut dapat disimpulkan bahwa Nilai y = 87, karena merujuk pada alamat *px yang merujuk pada variabel x

- ```

main() {
 double *pu, nu;
 double u = 1234.0;

 pu = &u;
 nu = *pu;

 printf("Alamat dari u = %p\n", &u);
 printf("Isi pu = %p\n", pu);
 printf("Isi u = %lf\n", u);
 printf("Nilai yang ditunjuk oleh pu = %f\n", *pu);
 printf("Nilai nu = %f\n", nu);

```

```
}

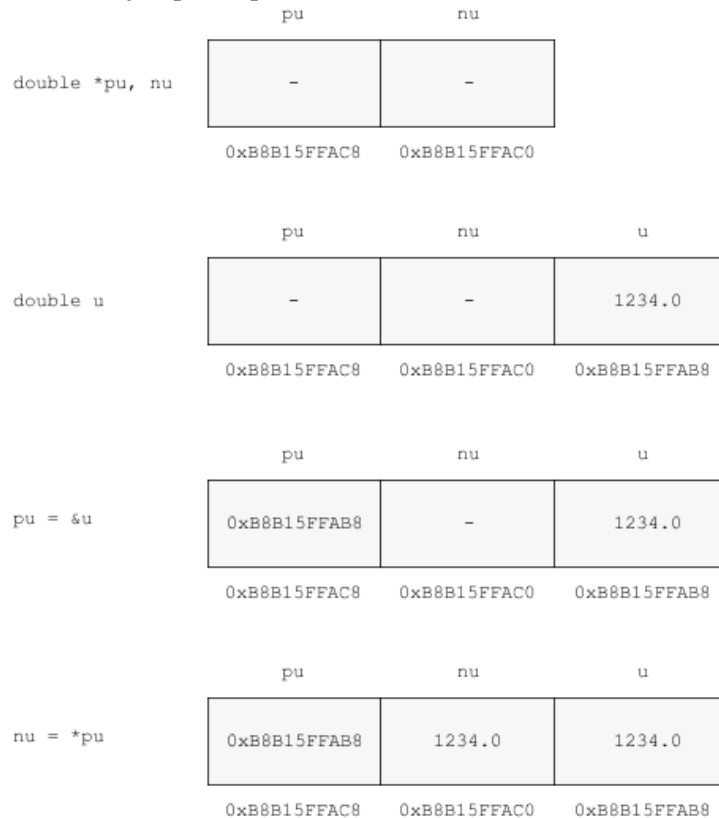
```

**Jawab :**

a. Ilustrasi alokasi memori

Alur :

1. Inisialisasi double \*pu dan nu
2. Inisialisasi double u yang berisi 1234.0
3. Pointer pu mengarahkan alamat ke u
4. Double nu merujuk pada \*pu



Ilustrasi alokasi memori

b. Perkiraan hasil eksekusi

Berikut adalah hasil eksekusi pointer

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
orkshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1> & .\soal 2.exe
Alamat dari u = 0000007E55FFDE8
Isi pu = 0000007E55FFDE8
Isi u = 1234.000000
Nilai yang ditunjuk oleh pu = 1234.000000
Nilai nu = 1234.000000
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1>

```

Gambar hasil eksekusi pointer

Berdasarkan gambar tersebut dapat disimpulkan bahwa Nilai nu = 1234.000000, karena merujuk pada alamat \*pu yang merujuk pada variabel u

```

3. main(){
 float d = 54.5f, *pd;

 printf("Isi d mula-mula = %g\n", d);

 pd = &d;
 *pd += 10;

```

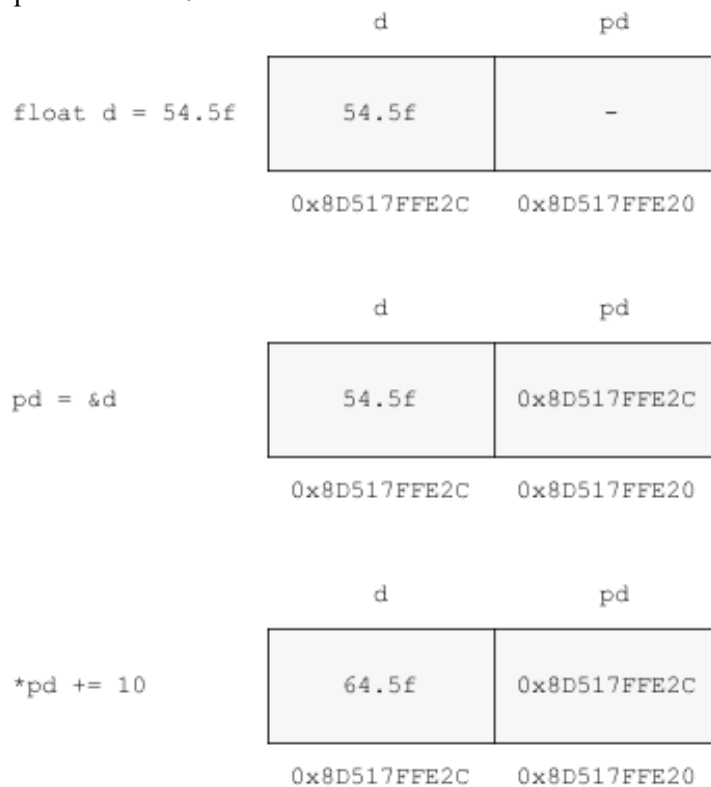
```
printf("Isi d sekarang = %g\n", d);
}
```

**Jawab :**

a. Ilustrasi alokasi memori

Alur :

1. Inisialisasi float d yang berisi 54.5f, \*pd
2. Pointer pd mengarahkan alamat ke float d
3. Nilai \*pd ditambah 10



Ilustrasi alokasi memori

b. Perkiraan hasil eksekusi

Berikut adalah hasil eksekusi pointer

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS C/C++ Compile Run - Pointer 1
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman
Struktur Data (P. Iqbal)\Praktikum 2> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\
Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman
Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1> & .\soal 3.exe
Isi d mula-mula = 54.5
Isi d sekarang = 64.5
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman
Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1>

```

Gambar hasil eksekusi pointer

Berdasarkan gambar tersebut, maka sekarang variabel float d berisi 64.5, karena pointer \*pd yang merujuk pada variabel float d ditambahkan dengan 10

```
4. main(){
 int z = 20, s = 30, *pz, *ps;
```

```

pz = &z;
ps = &s;
*pz += *ps;
printf("z = %d\n", z);
printf("s = %d\n", s);
}

```

**Jawab :**

a. Ilustrasi alokasi memori

Alur :

1. Inisialisasi variabel int z berisi 20, s berisi 30, pointer \*pz, dan pointer \*ps
2. Pointer pz mengarahkan alamat ke int z
3. Pointer ps mengarahkan alamat ke int s
4. Nilai \*pz ditambah merupakan penjumlahan dari \*pz dan \*ps

|                              |              |              |              |              |
|------------------------------|--------------|--------------|--------------|--------------|
|                              | z            | s            | pz           | ps           |
| int z = 20, s = 30, *pz, *ps | 20           | 30           | -            | -            |
|                              | 0x45F73FF8BC | 0x45F73FF8B8 | 0x45F73FF8B0 | 0x45F73FF8A8 |
|                              | z            | s            | pz           | ps           |
| pz = &z                      | 20           | 30           | 0x45F73FF8BC | -            |
|                              | 0x45F73FF8BC | 0x45F73FF8B8 | 0x45F73FF8B0 | 0x45F73FF8A8 |
|                              | z            | s            | pz           | ps           |
| ps = &s                      | 20           | 30           | 0x45F73FF8BC | 0x45F73FF8B8 |
|                              | 0x45F73FF8BC | 0x45F73FF8B8 | 0x45F73FF8B0 | 0x45F73FF8A8 |
|                              | z            | s            | pz           | ps           |
| *pz += *ps                   | 50           | 30           | 0x45F73FF8BC | 0x45F73FF8B8 |
|                              | 0x45F73FF8BC | 0x45F73FF8B8 | 0x45F73FF8B0 | 0x45F73FF8A8 |

Ilustrasi alokasi memori

b. Perkiraan hasil eksekusi

Berikut adalah hasil eksekusi pointer

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS C/C++ Compile Run - Pointer 1
Struktur Data (P. Iqbal)\Praktikum 2> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman
Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1> cd 'c:\Users\ilyas\OneDrive\Doc
uments\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Prakt
ikum 2\output\Pointer 1'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman
Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1> & .\soal 4.exe
z = 50
s = 30
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman
Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1>

```

Gambar hasil eksekusi pointer

Berdasarkan gambar tersebut, maka sekarang  $int\ z = 50$  sedangkan  $s$  tetap 30, karena variabel  $z$  merujuk pada pointer  $*pz$  yang nilainya merupakan penjumlahan dari  $*pz$  dan  $*ps$ , yang dimana  $*ps$  merujuk pada nilai  $int\ s$

```
5. main(){
 char c = 'Q', *cp = &c;

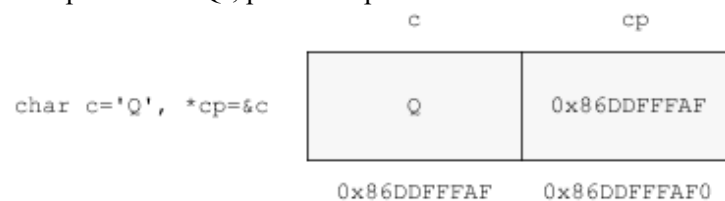
 printf("%c %c\n", c, *cp);
 c = '/';
 printf("%c %c\n", c, *cp);
 *cp = '(';
 printf("%c %c\n", c, *cp);
}
```

**Jawab :**

a. Ilustrasi alokasi memori

Alur :

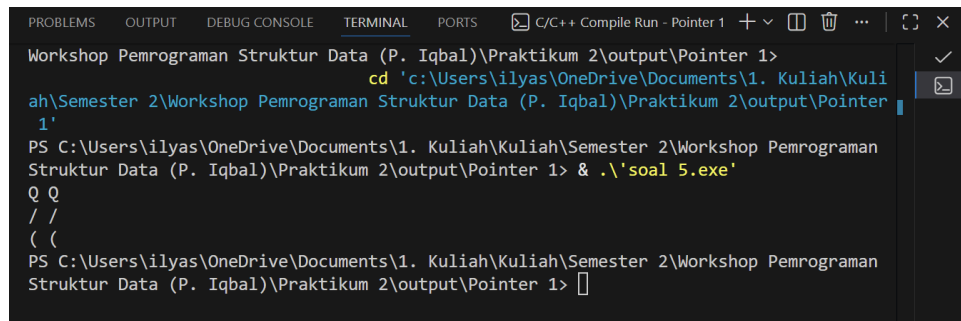
1. Inisialisasi  $char\ c = 'Q'$ , pointer  $*cp$  mengarahkan alamat ke  $int\ c$
2. Tampilkan  $c = 'Q'$ , pointer  $*cp$
3. Ubah nilai  $c = '/'$
4. Tampilkan  $c = 'Q'$ , pointer  $*cp$
5. Ubah nilai  $c = '('$
6. Tampilkan  $c = 'Q'$ , pointer  $*cp$



Ilustrasi alokasi memori

b. Perkiraan hasil eksekusi

Berikut adalah hasil eksekusi pointer



```
Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1>
cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman
Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman
Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1> & .\'soal 5.exe'
Q Q
/
(
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman
Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1>
```

Gambar hasil eksekusi pointer

Berdasarkan gambar tersebut, maka terjadi perubahan sebanyak 3x pada nilai char c dan pointer \*cp yang merujuk pada char c. Pertama berisikan 'Q', kemudian pada baris berikutnya nilai char c diubah menjadi '/' yang otomatis \*cp juga menjadi '/', kemudian baris berikutnya pointer \*cp diubah menjadi '(' yang otomatis nilai char c menjadi '('

```
6. main() {
 int x = 1, y = 2, *ip;

 ip = &x;
 y = *ip;
 *ip = 3;

 printf("x = %d, y = %d", x, y);
}
```

**Jawab :**

a. Ilustrasi alokasi memori

Alur :

1. Inisialisasi int x = 1, y = 2, dan pointer \*ip
2. Pointer ip mengarahkan alamat pada int x
3. Int y merujuk pada alamat pointer \*ip
4. Pointer \*ip bernilai 3

### Ilustrasi alokasi memori

- ```

PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Strukt
ur Data (P. Iqbal)\Praktikum 2> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semes
ter 2\Workshop Pemrograman Struktural Data (P. Iqbal)\Praktikum 2\output\Pointer 1'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Strukt
ur Data (P. Iqbal)\Praktikum 2\output\Pointer 1> & .\'soal 6.exe'
x = 3, y = 1
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Strukt
ur Data (P. Iqbal)\Praktikum 2\output\Pointer 1>

```

Berdasarkan gambar tersebut, maka nilai $x = 3$, karena pointer $*ip$ mengarah pada int x , ketika $*ip$ berubah nilainya menjadi 3 maka nilai $x = 3$, dan nilai $y = 1$ karena variabel y merujuk pada $*ip$ sebelum diubah yang mana $*ip$ menyimpan alamat int x yang berisi 1, maka nilai $y = 1$

Jawab :

- 11

Alur :

1. Inisialisasi variabel int i1, i2, pointer *p1, dan *p2
2. Int i1 bernilai 9
3. Pointer *p1 mengarahkan alamat pada int i1
4. Int i2 bernilai yang merujuk pada $*i1 / 2 - 2 * 3$
5. Pointer p2 merujuk pada p1

	i1	i2	p1	p2
int i1, i2, *p1, *p2	-	-	-	-
	0x8FA1BFFD8C	0x8FA1BFFD88	0x8FA1BFFD80	0x8FA1BFFD78
	i1	i2	p1	p2
i1 = 9	9	-	-	-
	0x8FA1BFFD8C	0x8FA1BFFD88	0x8FA1BFFD80	0x8FA1BFFD78
	i1	i2	p1	p2
p1 = &i1	9	-	0x8FA1BFFD8C	-
	0x8FA1BFFD8C	0x8FA1BFFD88	0x8FA1BFFD80	0x8FA1BFFD78
	i1	i2	p1	p2
i2 = *p1 / 2 - 2 * 3	9	-2	0x8FA1BFFD8C	-
	0x8FA1BFFD8C	0x8FA1BFFD88	0x8FA1BFFD80	0x8FA1BFFD78
	i1	i2	p1	p2
p2 = p1	9	-2	0x8FA1BFFD8C	0x8FA1BFFD8C
	0x8FA1BFFD8C	0x8FA1BFFD88	0x8FA1BFFD80	0x8FA1BFFD78

Ilustrasi alokasi memori

b. Perkiraan hasil eksekusi

Berikut adalah hasil eksekusi pointer

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS C/C++ Compile Run - Pointer 1
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman
Struktur Data (P. Iqbal)\Praktikum 2> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\
Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman
Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1> & .\soal 7.exe
i1=9,i2=-2,*p1=9,*p2=9
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman
Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1>

```

Gambar hasil eksekusi pointer

Berdasarkan gambar tersebut, maka nilai

- i1 = 9, nilai i2 = 2 karena merujuk pada *p1 yang bernilai 9 kemudian dioperasikan $/ 2 - 2 * 3 = 2$,
- *p1 = 9 karena meyimpan alamat int i1, dan *p2 = 9 karena merujuk pada *p2

```

8. main() {
    int count = 10, *temp, sum = 7;

```

```

temp = &count;
*temp = 32;
temp = &sum;
*temp = count;
sum = *temp * 4;

printf("count=%d, *temp=%d, sum=%d\n", count,*temp, sum );
}

```

Jawab :

a. Ilustrasi alokasi memori

Alur :

1. Inisialisasi int count = 10, pointer *temp, dan int sum = 7
2. Pointer temp mengarahkan alamat ke int count
3. Pointer *temp bernilai = 32
4. Pointer temp mengarahkan alamat ke int sum
5. Pointer *temp merujuk pada int count
6. Int sum bernilai = *temp * 4

	count	temp	sum
int count = 10, *temp, sum = 7	10	-	7
	0xD5091FFB3C	0xD5091FFB30	0xD5091FFB2C

	count	temp	sum
temp = &count	10	0xD5091FFB3C	7
	0xD5091FFB3C	0xD5091FFB30	0xD5091FFB2C

	count	temp	sum
*temp = 32	32	0xD5091FFB3C	7
	0xD5091FFB3C	0xD5091FFB30	0xD5091FFB2C

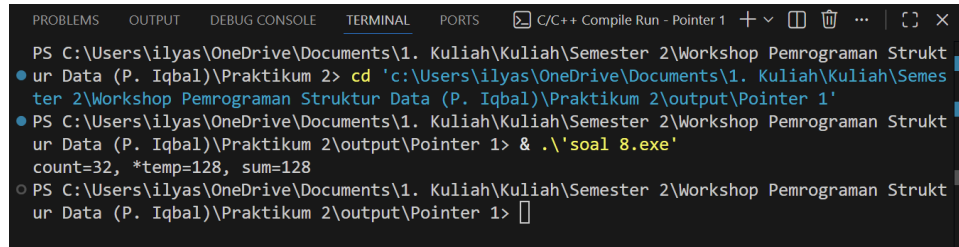
	count	temp	sum
temp = &sum	32	0xD5091FFB2C	7
	0xD5091FFB3C	0xD5091FFB30	0xD5091FFB2C

	count	temp	sum
*temp = count	32	0xD5091FFB2C	32
	0xD5091FFB3C	0xD5091FFB30	0xD5091FFB2C

	count	temp	sum
sum = *temp * 4	32	0xD5091FFB2C	128
	0xD5091FFB3C	0xD5091FFB30	0xD5091FFB2C

Ilustrasi alokasi memori

b. Perkiraan hasil eksekusi



```
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1> & .\soal 8.exe
count=32, *temp=128, sum=128
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\output\Pointer 1> 
```

Berdasarkan gambar tersebut, maka nilai

- `count = 32` karena `*temp` mengarahkan alamat ke `int count` dimana `*temp` bernilai `32`,
- `*temp = 128` karena `*temp` mengarahkan alamat untuk yang keduanya ke `int sum` dimana `int sum` berisi `*temp * 4` ($32 * 4 = 128$),
- `sum = 128` karena merupakan hasil dari operasi `*temp * 4` ($32 * 4 = 128$)

```
9. main(){
    int count = 13, sum = 9, *x, *y;

    x = &count;
    *x = 27;
    y = x;
    x = &sum;
    *x = count;
    sum = *x / 2 * 3;

    printf("count=%d, sum=%d, *x=%d, *y=%d\n", count, sum, *x, *y);
}
```

Jawab :

a. Ilustrasi alokasi memori

Alur :

1. Inisialisasi `int count = 13`, `sum 9`, pointer `*x`, `*y`
2. Pointer `x` mengarahkan alamat ke `int count`
3. Pointer `*x` bernilai `27`
4. `Int y` bernilai `= int x`
5. Pointer `x` mengarahkan alamat ke `int sum`
6. Pointer `*x` bernilai `= int count`
7. `Int sum` bernilai `= pointer *x / 2 * 3`

	count	sum	x	y
int count = 13, sum = 9, *x, *y	13	9	-	-
	0xD5091FFB3C	0xD5091FFB30	0xD5091FFB2C	0xD5091FFB2C
	count	sum	x	y
x = &count	13	9	0xD5091FFB3C	-
	0xD5091FFB3C	0xD5091FFB30	0xD5091FFB2C	0xD5091FFB2C
	count	sum	x	y
*x = 27	27	9	0xD5091FFB3C	-
	0xD5091FFB3C	0xD5091FFB30	0xD5091FFB2C	0xD5091FFB2C
	count	sum	x	y
	27	9	0xD5091FFB3C	0xD5091FFB3C
	0xD5091FFB3C	0xD5091FFB30	0xD5091FFB2C	0xD5091FFB2C
	count	sum	x	y
x = &sum	27	9	0xD5091FFB30	0xD5091FFB3C
	0xD5091FFB3C	0xD5091FFB30	0xD5091FFB2C	0xD5091FFB2C
	count	sum	x	y
*x = count	27	9	0xD5091FFB30	0xD5091FFB3C
	0xD5091FFB3C	0xD5091FFB30	0xD5091FFB2C	0xD5091FFB2C
	count	sum	x	y
sum = *x / 2 * 3	27	39	0xD5091FFB30	0xD5091FFB3C
	0xD5091FFB3C	0xD5091FFB30	0xD5091FFB2C	0xD5091FFB2C

Ilustrasi alokasi memori

b. Perkiraan hasil eksekusi

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS C/C++ Compile Run - Pointer 1
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Strukt
ur Data (P. Iqbal)\Praktikum 2> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Sem
ter 2\Workshop Pemrograman Struktural Data (P. Iqbal)\Praktikum 2\output\Pointer 1'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Strukt
ur Data (P. Iqbal)\Praktikum 2\output\Pointer 1> & .\soal 9.exe
count=27, sum=39, *x=39, *y=27
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Strukt
ur Data (P. Iqbal)\Praktikum 2\output\Pointer 1>

```

Berdasarkan gambar tersebut, maka nilai :

- count = 27 karena alamat count disimpan oleh pointer *x yang bernilai = 27,
- sum = 39 karena alamat sum disimpan oleh pointer *x yang semula bernilai 27 kemudian dilakukan operasi $*x / 2 * 3$ ($27/2*3$) = 39,
- *x = 39 karena semua *x menyimpan alamat count yang berisi 13, kemudian *x diberi nilai 27, kemudian *x menyimpan alamat sum dimana setelahnya sum melakukan operasi $*x / 2 * 3$ ($27/2*3$) = 39,
- *y = 27 karena y merujuk pada *x dimana saat itu x bernilai 27

```

10. int r, q = 7;
    int go_crazy(int *, int *);

    main() {
        int *ptr1 = &q;
        int *ptr2 = &q;

        r = go_crazy(ptr1, ptr2);
        printf("q=%d, r=%d, *ptr1=%d, *ptr2=%d\n", q, r, *ptr1, *ptr2);

        ptr2 = &r;

        go_crazy(ptr2, ptr1);
        printf("q=%d, r=%d, *ptr1=%d, *ptr2=%d\n", q, r, *ptr1, *ptr2);
    }

    int go_crazy(int *p1, int *p2){
        int x = 5;

        r = 12;
        *p2 = *p1 * 2;
        p1 = &x;
        return *p1 * 3;
    }

```

Jawab :

a. Ilustrasi alokasi memori

Alur :

Bagian 1: Pemanggilan Pertama `r = go_crazy(ptr1, ptr2)`

1. Kondisi awal: `q = 7, r = 0`. `ptr1` dan `ptr2` keduanya menunjuk ke `q`.
2. Masuk ke fungsi: Parameter `p1` menerima alamat `q`, `p2` juga menerima alamat `q`.
3. Inisialisasi lokal: Variabel `x` dibuat di memori dengan nilai 5.
4. Update Global: Variabel global `r` diubah nilainya menjadi 12.
5. Operasi Pointer: `*p2 = *p1 * 2`. Karena `p1` dan `p2` menunjuk `q`, maka `q = 7 * 2`. Sekarang `q` bernilai 14.
6. Perpindahan Alamat: `p1 = &x`. Sekarang pointer `p1` tidak lagi menunjuk `q`, tapi menunjuk ke variabel lokal `x`.
7. Return Nilai: Fungsi mengembalikan `*p1 * 3` ($5 \times 3 = 15$). Nilai 15 ini disimpan ke variabel global `r` (menimpa angka 12 tadi).

Bagian 2: Pemanggilan Kedua `go_crazy(ptr2, ptr1)`

1. Masuk ke fungsi: Parameter `p1` menerima alamat `ptr2` (yaitu alamat `r`), `p2` menerima alamat `ptr1` (yaitu alamat `q`).
2. Hati-hati: Di sini posisi `p1` dan `p2` tertukar dibanding pemanggilan pertama.
3. Inisialisasi lokal: Variabel `x` baru dibuat lagi dengan nilai 5.
4. Update Global: Variabel global `r` diatur ulang menjadi 12.
5. Operasi Pointer: `*p2 = *p1 * 2`.
6. `p2` menunjuk ke `q` (isinya 14).
7. `p1` menunjuk ke `r` (isinya 12).

8. Maka: $q = 12 * 2$. Sekarang q bernilai 24.
9. Perpindahan Alamat: $p1 = \&x$. Pointer lokal $p1$ sekarang menunjuk ke variabel x yang bernilai 5.
10. Selesai Fungsi: Fungsi mengembalikan 15, tapi karena di main tidak ada variabel yang menampung (tidak ada $r = \dots$), maka hasil 15 ini dibuang.



Ilustrasi alokasi memori

b. Perkiraan hasil eksekusi

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
P5 C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 1> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 1\output'
P5 C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 1\output> & .\soal 10.exe
q=14, r=15, *ptr1=14, *ptr2=14
q=24, r=12, *ptr1=24, *ptr2=12
P5 C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 1\output>

```

Berdasarkan gambar tersebut, maka nilai :

- $q = 24$ karena pada pemanggilan fungsi kedua, $*p2$ merujuk pada alamat q , di mana q diisi dari hasil operasi $*p1 * 2$ ($12 \times 2 = 24$).

- $r = 12$ karena pada pemanggilan fungsi kedua, variabel global r diatur ulang menjadi 12 di dalam fungsi, dan hasil return fungsi (15) tidak disimpan kembali ke dalam r (panggilan fungsinya berdiri sendiri tanpa $r = \dots$).
- $*ptr1 = 24$ karena $ptr1$ menyimpan alamat variabel q , sehingga nilainya akan selalu mengikuti perubahan terbaru pada isi variabel q .
- $*ptr2 = 12$ karena setelah baris $ptr2 = \&r$, pointer ini menyimpan alamat variabel r , sehingga nilainya mengikuti perubahan terbaru pada isi variabel r yaitu 12.

D. KESIMPULAN

Berdasarkan hasil praktikum terhadap sepuluh percobaan yang telah dilakukan, dapat disimpulkan bahwa pemahaman mengenai konsep pointer dan alamat memori merupakan aspek fundamental dalam pemrograman bahasa C untuk mengelola data secara lebih efisien dan langsung. Melalui analisis alokasi memori pada setiap baris pernyataan, terbukti bahwa pointer bukan sekadar menyimpan nilai biasa, melainkan menyimpan alamat memori variabel lain yang memungkinkan manipulasi data secara tidak langsung melalui operator dereference (*) dan address-of (&).

Praktikum ini juga memperlihatkan bagaimana pointer dapat berpindah rujukan secara dinamis, mengelola berbagai tipe data mulai dari integer hingga float, serta bagaimana fungsi seperti `go_crazy` dapat berinteraksi dengan variabel global dan parameter alamat (pass by reference). Secara keseluruhan, penerapan pointer dalam praktikum ini memperkuat pemahaman terhadap logika pengelolaan memori, efisiensi penggunaan sumber daya melalui alamatnya, serta cara berpikir sistematis dalam menyelesaikan permasalahan pemrograman yang lebih kompleks dan profesional.

LAPORAN PRAKTIKUM 1

Pointer 2



Disusun Oleh:

Ilyas Rahman Salim (5325600063)

Program Studi D4 Teknologi Rekayasa Multimedia

Departemen Teknologi Multimedia Kreatif

Politeknik Elektronika Negeri Surabaya

2026

A. TUJUAN PEMBELAJARAN

1. Memahami tentang Pointer to Array
2. Memahami tentang Pointer to String

B. DASAR TEORI

Pointer dan Array (*pointer to array*)

Hubungan antara pointer dan array pada C sangatlah erat. Sebab sesungguhnya array secara internal akan diterjemahkan dalam bentuk pointer. Pembahasan berikut akan memberikan gambaran hubungan antara pointer dan array. Misalnya dideklarasikan di dalam suatu fungsi

```
static int tgl_lahir[3] = { 01, 09, 64 };
```

dan

```
int *ptgl;
```

Kemudian diberikan instruksi

```
ptgl = &tgl_lahir[0]; //pointer to array of integer
```

maka **ptgl** akan berisi alamat dari elemen array **tgl_lahir** yang berindeks nol. Instruksi di atas bisa juga ditulis menjadi

```
ptgl = tgl_lahir;
```

sebab nama array tanpa tanda kurung menyatakan alamat awal dari array. Sesudah penugasan seperti di atas,

```
*ptgl
```

dengan sendirinya menyatakan elemen pertama (berindeks sama dengan nol) dari array **tgl_lahir**.

Pointer dan String (*pointer to string*)

Contoh hubungan pointer dan string ditunjukkan pada program berikut.

```
//Program : ptr4.c
```

```
#include <stdio.h>
```

```
main() {
```

```
    //pkota menunjuk konstanta string "SEMARANG"
```

```
    char *pkota = "SEMARANG";
```

```
    printf("String yang ditunjuk oleh pkota = ");
```

```
    puts(pkota);          // printf("%s\n", pkota);
```

```
}
```

Contoh eksekusi :

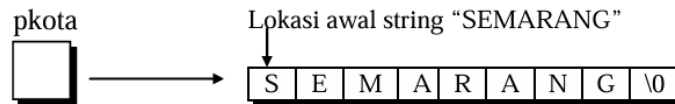
String yang ditunjuk oleh pkota = SEMARANG

Pada program di atas,

```
char *pkota = "SEMARANG";
```

akan menyebabkan kompiler

- mengalokasikan variabel `pkota` sebagai variabel pointer yang menunjuk ke obyek bertipe `char` dan menempatkan konstanta "SEMARANG" dalam suatu memori
- kemudian pointer `pkota` akan menunjuk ke lokasi string "SEMARANG".



Gambar 20.1 Pointer menunjuk data **string**

Pernyataan di atas menyerupai pernyataan

```
char kota[] = "SEMARANG";
```

tetapi sebenarnya kedua pernyataan inisialisasi di depan tidaklah tepat sama. Sebab `pkota` adalah pointer (menyatakan alamat) yang dengan mudah dapat diatur agar menunjuk ke string lain (bukan string "SEMARANG"), sedangkan `kota` adalah array (array menyatakan alamat yang konstan, tak dapat diubah).

Jadi, ada beberapa cara penulisan ekspresi variabel yang menghasilkan sebuah address memory, di antaranya adalah :

1. Menuliskan ampersand di depan sebuah variabel 'normal'
→ Misalnya `a` dideklarasikan sebagai sebuah variabel normal, maka `&a` akan menghasilkan address memory dari variabel `a` tersebut
2. Menuliskan nama dari sebuah variabel pointer
→ Misalnya `a` dideklarasikan sebagai variabel sebagai `int *a`, maka `a` akan berisi sebuah address memory yang tersimpan pada variabel `a` (setelah ada sebuah address memory yang di-assign ke variabel `a` tersebut)
3. Menuliskan nama sebuah variabel array tanpa indexnya
→ Misalnya dideklarasikan
 - `float nilai[5]`, maka penulisan `nilai` artinya sama dengan `&nilai[0]`
 - `int b[3][4]`, maka `b[3]` adalah sama dengan `&b[3][0]` dan `b[0]` adalah sama dengan `&b[0][0]`.

Bisa disimpulkan bahwa sebuah variabel array yang dituliskan tanpa kurung siku indexnya adalah sama dengan address memory dari elemen pertama array tersebut

C. PERCOBAAN

Untuk setiap program pada no 1 – 8 di bawah ini,

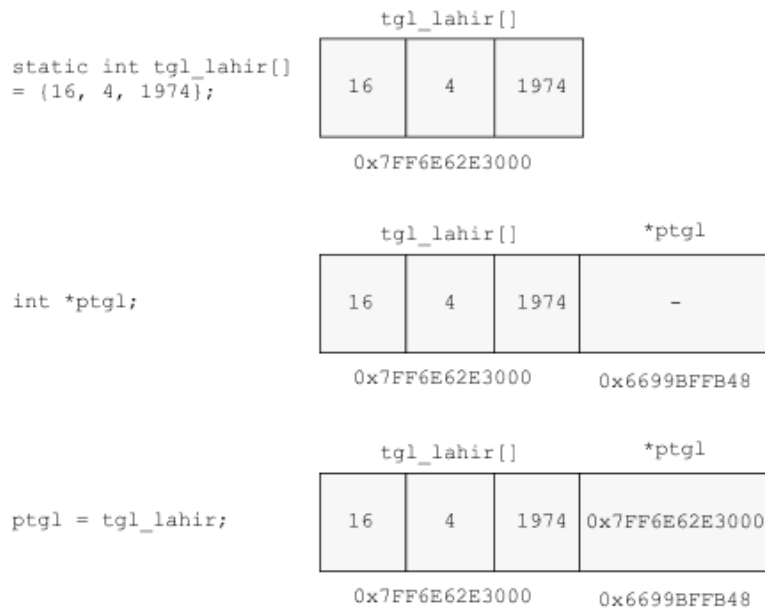
- gambarkan ilustrasi alokasi memori dari setiap baris pernyataan yang diproses
- perkirakan hasil eksekusinya

Untuk no. 9 buatlah programnya

1. Hubungan antara pointer dan array. Suatu nama array yang ditulis tanpa dengan indeksinya menunjukkan alamat elemen pertama dari array (versi 1).

```
#include <stdio.h>

main(){
    static int tgl_lahir[] = {16, 4, 1974};
    int *ptgl;
    ptgl = tgl_lahir;
    printf("Nilai yang ditunjuk oleh ptgl = %d\n", *ptgl);
    printf("Nilai dari tgl_lahir[0] = %d\n", tgl_lahir[0]);
}
```



```

● PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pem
rograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output> & .\'soal 1.e
xe'
Nilai yang ditunjuk oleh ptgl = 16
Nilai dari tgl_lahir[0] = 16
○ PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pem
rograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output>

```

2. Hubungan antara pointer dan array. Suatu nama array yang ditulis tanpa dengan indeksinya menunjukkan alamat elemen pertama dari array (versi 2).

```

#include <stdio.h>

main(){
    static int tgl_lahir[] = {16, 4, 1974};
    int *ptgl, i;

    ptgl = tgl_lahir;

    printf("Nilai yang ditunjuk oleh ptgl = %d\n", *ptgl);
    for (i=0; i<3; i++)
        printf("Nilai dari tgl_lahir[i] = %d\n", *(ptgl+i));
}

```



```

PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pem
rograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output> & .\'soal 2.e
xe'
Nilai yang ditunjuk oleh ptgl = 16
Nilai dari tgl_lahir[i] = 16
Nilai dari tgl_lahir[i] = 4
Nilai dari tgl_lahir[i] = 1974
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pem
rograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output>

```

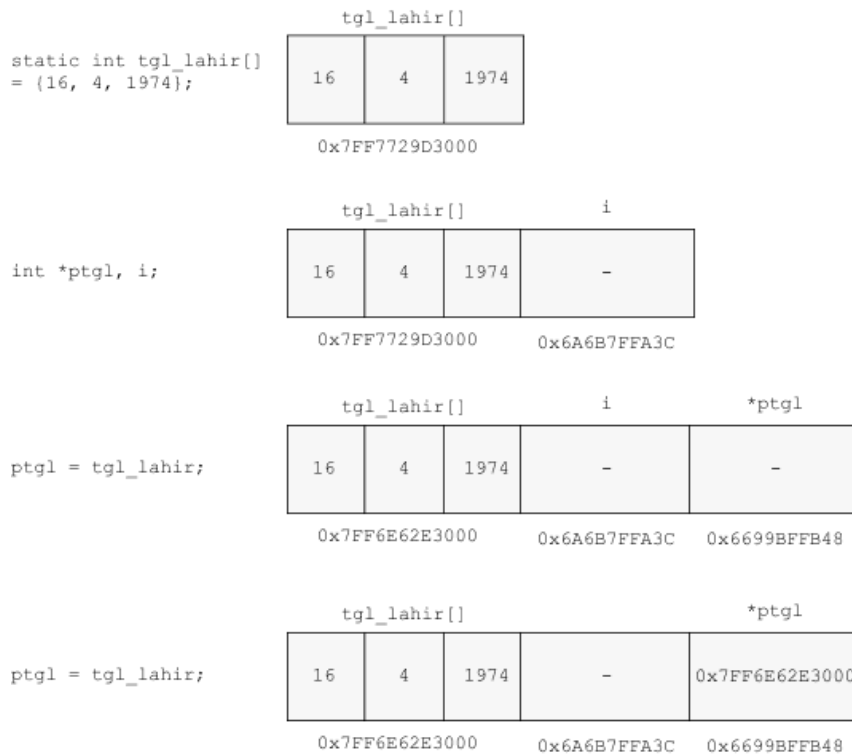
3. Hubungan antara pointer dan array. Suatu nama array yang ditulis tanpa dengan indeksinya menunjukkan alamat elemen pertama dari array (versi 3).

```

#include <stdio.h>

main(){
    static int tgl_lahir[] = {16, 4, 1974};
    int i;
    int *ptgl;
    ptgl = tgl_lahir;
    printf("Nilai yang ditunjuk oleh ptgl = %d\n", *ptgl);
    for (i=0; i<3; i++)
        printf("Nilai dari tgl_lahir[i] = %d\n", *ptgl++);
}

```



```
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman
• Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output> & .\'soal 3.exe'
Nilai yang ditunjuk oleh ptgl = 16
Nilai dari tgl_lahir[i] = 16
Nilai dari tgl_lahir[i] = 4
Nilai dari tgl_lahir[i] = 1974
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman
○ Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output> □
```

Analisislah dan jelaskan perbedaan antara aplikasi pada nomor 1, 2 dan 3

4. Menukarkan isi 2 string tanpa pemakaian pointer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define PANJANG 20
```

```
char nama1[PANJANG] = "AHMAD";
```

```
char nama2[PANJANG] = "RIFDA";
```

```
main(){
```

```
    char namax[PANJANG];
```

```
    puts("SEMULA : ");
```

```
    printf("nama1 --> %s\n", nama1);
```

```
printf("nama2 --> %s\n", nama2);
```

```
strcpy(namax, nama1);
```

```
strcpy(nama1, nama2);
```

```
strcpy(nama2, namax);
```

```
puts("KINI : ");
```

```
printf("nama1 --> %s\n", nama1);
```

```
printf("nama2 --> %s\n", nama2);
```

```
}
```



```

PS C:\Users\ilycd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Point
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output> & .
SEMULA :
nama1 --> AHMAD
nama2 --> RIFDA
KINI :
nama1 --> RIFDA
nama2 --> RIFDA
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output>

```

5. Menukarkan isi 2 string dengan fasilitas pointer

```
#include <stdio.h>
```

```
char *nama1 = "AHMAD";
```

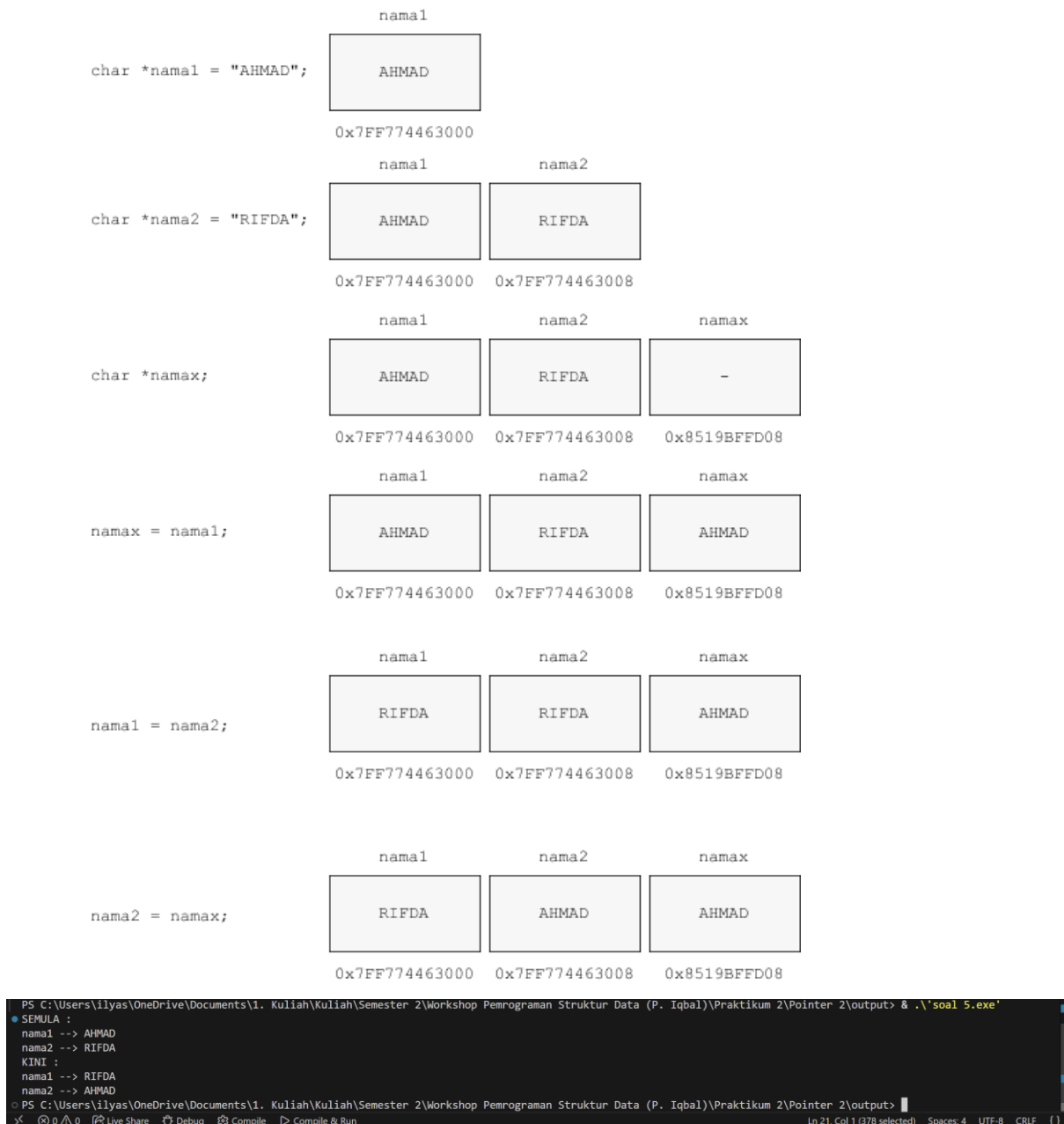
```
char *nama2 = "RIFDA";
```

```
main(){
    char *namax;

    puts("SEMULA : ");
    printf("nama1 --> %s\n", nama1);
    printf("nama2 --> %s\n", nama2);

    namax = nama1;
    nama1 = nama2;
    nama2 = namax;

    puts("KINI : ");
    printf("nama1 --> %s\n", nama1);
    printf("nama2 --> %s\n", nama2);
}
```



Analisislah dan jelaskan perbedaan antara aplikasi pada nomor 4 dengan nomor 5

- Penggunaan variable index pada array dan variable index pada pointer, untuk menunjuk suatu nilai data di dalam suatu variable array. Berikan analisis dan kesimpulan

```

main(){
    int nilai[10]={86,75,98,66,56,76,80,95,70,60};
    int index, *ip;

    printf("Mencetak menggunakan array\n");
    printf("Daftar nilai siswa\n\n");
  
```

```

for(index=0; index<10; index++)
    printf("%3d",nilai[index]);

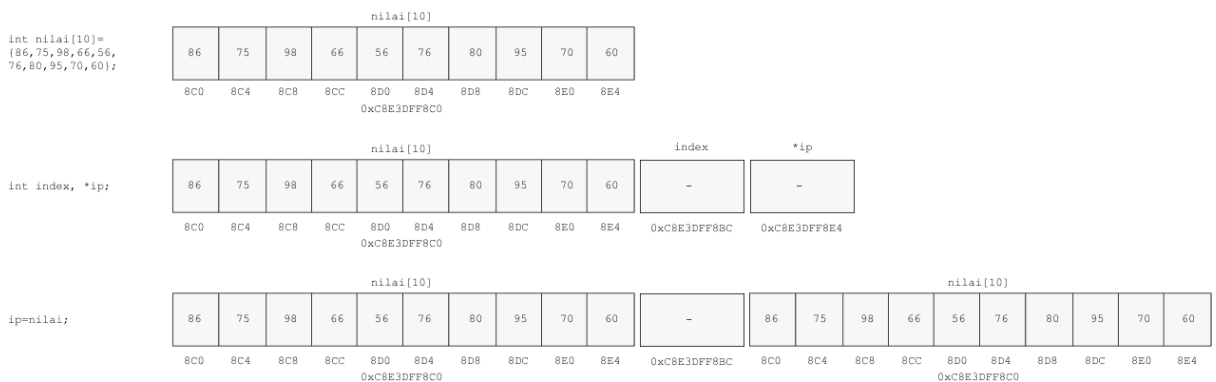
puts("\n");
printf("Mencetak menggunakan pointer dan index\n");
printf("Daftar nilai siswa\n\n");

for(index=0; index<10; index++)
    printf("%3d",*(nilai+index));

puts("\n");
printf("Mencetak menggunakan pointer\n");
printf("Daftar nilai siswa\n\n");

ip=nilai;
for(index=0; index<10; index++)
    printf("%3d",*ip++);
}

```



```

PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pem
rograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output> & .\soal 6.e
xe'
Mencetak menggunakan array
Daftar nilai siswa

 86 75 98 66 56 76 80 95 70 60

Mencetak menggunakan pointer dan index
Daftar nilai siswa

 86 75 98 66 56 76 80 95 70 60

Mencetak menggunakan pointer
Daftar nilai siswa

 86 75 98 66 56 76 80 95 70 60
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pem
rograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output>

```

7. Berikan ilustrasi dan jelaskan apa yang dilakukan oleh potongan program di bawah ini

```
main() {
    char *text_pointer = "Good morning!";

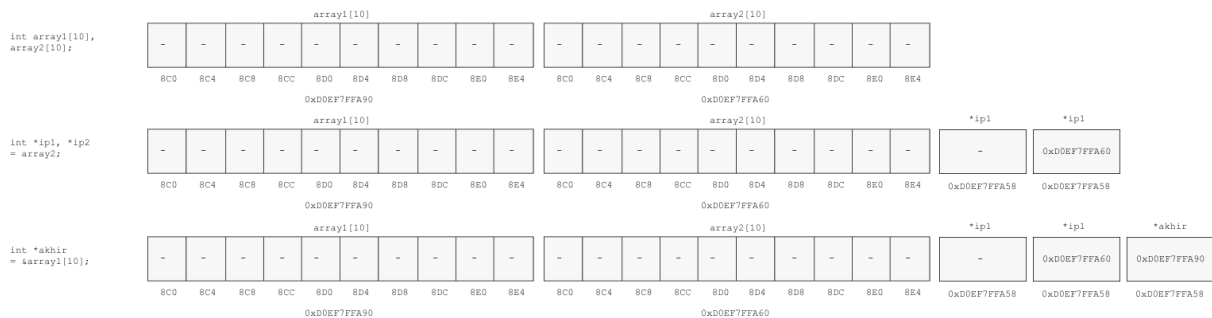
    for( ; *text_pointer != '\0'; ++text_pointer)
        printf("%c", *text_pointer);
}
```



```
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output> & .\soal 7.exe
Good morning!
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output>
```

8. Berikan ilustrasi dan jelaskan apa yang dilakukan oleh potongan program di bawah ini

```
int array1[10], array2[10];
int *ip1, *ip2 = array2;
int *akhir = &array1[10];
for(ip1 = &array1[0]; ip1 < akhir; ip1++)
    *ip2++ = *ip1;
```



9. Definisikan sebuah fungsi untuk membaca sebuah array (dengan tipe sembarang) menggunakan pointer. Buatlah program untuk membaca array tersebut dalam rangka mencari sebuah nilai tertentu dan laporkan hasilnya berhasil menemukan atau tidak

a. Source Code

```
#include <stdio.h>

int cariNilai(int *ptr, int ukuran, int target) {
    for (int i = 0; i < ukuran; i++) {
```

```

        if (*(ptr + i) == target) {
            return 1;
        }
    }
    return 0;
}

int main() {
    int data[] = {10, 25, 30, 45, 50, 65, 80};
    int n = sizeof(data) / sizeof(data[0]);
    int cari;

    printf("=== Program Pencarian Array dengan Pointer (C) ===\n");
    printf("Daftar Angka: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", data[i]);
    }

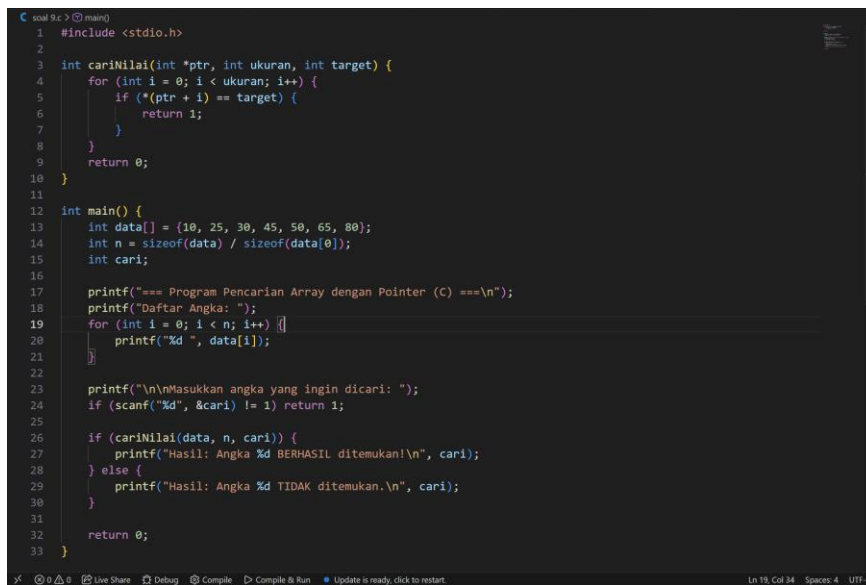
    printf("\n\nMasukkan angka yang ingin dicari: ");
    if (scanf("%d", &cari) != 1) return 1;

    if (cariNilai(data, n, cari)) {
        printf("Hasil: Angka %d BERHASIL ditemukan!\n", cari);
    } else {
        printf("Hasil: Angka %d TIDAK ditemukan.\n", cari);
    }

    return 0;
}

```

b. Screenshoot



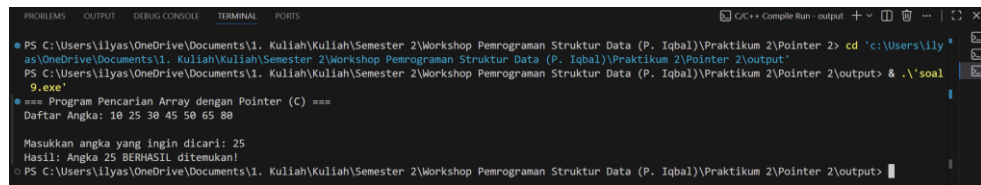
```

1  #include <stdio.h>
2
3  int cariNilai(int *ptr, int ukuran, int target) {
4      for (int i = 0; i < ukuran; i++) {
5          if (*(ptr + i) == target) {
6              return 1;
7          }
8      }
9      return 0;
10 }
11
12 int main() {
13     int data[] = {10, 25, 30, 45, 50, 65, 80};
14     int n = sizeof(data) / sizeof(data[0]);
15     int cari;
16
17     printf("=== Program Pencarian Array dengan Pointer (C) ===\n");
18     printf("Daftar Angka: ");
19     for (int i = 0; i < n; i++) {
20         printf("%d ", data[i]);
21     }
22
23     printf("\n\nMasukkan angka yang ingin dicari: ");
24     if (scanf("%d", &cari) != 1) return 1;
25
26     if (cariNilai(data, n, cari)) {
27         printf("Hasil: Angka %d BERHASIL ditemukan!\n", cari);
28     } else {
29         printf("Hasil: Angka %d TIDAK ditemukan.\n", cari);
30     }
31
32     return 0;
33 }

```

c. Output

Berikut adalah output tampilan ketika pencarian nilai pada array berhasil ditemukan



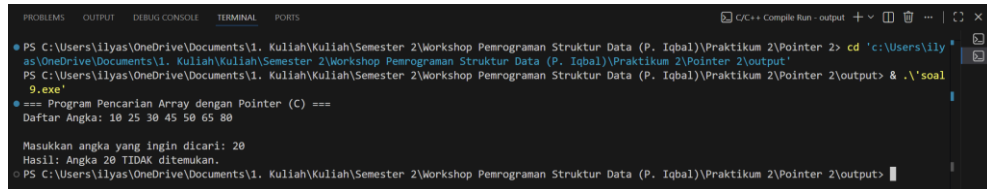
```
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output> & .\soal9.exe

=== Program Pencarian Array dengan Pointer (C) ===
Daftar Angka: 10 25 30 45 50 65 80

Masukkan angka yang ingin dicari: 25
Hasil: Angka 25 BERHASIL ditemukan!
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output>
```

Screenshoot ketika nilai berhasil dicari

Berikut adalah output tampilan ketika pencarian nilai pada array tidak berhasil ditemukan



```
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output> & .\soal9.exe

=== Program Pencarian Array dengan Pointer (C) ===
Daftar Angka: 10 25 30 45 50 65 80

Masukkan angka yang ingin dicari: 20
Hasil: Angka 20 TIDAK ditemukan.
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 2\output>
```

Screenshoot ketika nilai tidak berhasil dicari

D. KESIMPULAN

Berdasarkan seluruh rangkaian percobaan, dapat disimpulkan bahwa pointer dan array memiliki hubungan yang sangat erat namun berbeda dalam fleksibilitas, di mana nama array bertindak sebagai alamat konstan elemen pertamanya, sementara pointer adalah variabel dinamis yang dapat memanipulasi data melalui aritmatika alamat (seperti `*ptgl++` atau `*(ptgl+i)`). Praktikum ini membuktikan bahwa manipulasi data melalui pointer jauh lebih efisien dibandingkan penyalinan fisik data, seperti terlihat pada perbandingan penukaran string menggunakan `strcpy()` (menyalin karakter demi karakter) versus penukaran alamat pointer (hanya memindahkan referensi alamat), yang secara drastis mengurangi beban komputasi. Melalui implementasi double pointer, array of pointers, hingga fungsi pencarian array, terbukti bahwa penguasaan terhadap operator dereference (`*`) dan address-of (`&`) memungkinkan programmer untuk mengelola alokasi memori secara presisi, melakukan operasi pass-by-reference untuk mengubah data lintas fungsi, serta membangun struktur kode yang lebih sistematis dan performan untuk aplikasi profesional.

LAPORAN PRAKTIKUM 1

Pointer 3



Disusun Oleh:

Ilyas Rahman Salim (5325600063)

Program Studi D4 Teknologi Rekayasa Multimedia

Departemen Teknologi Multimedia Kreatif

Politeknik Elektronika Negeri Surabaya

2026

A. TUJUAN PEMBELAJARAN

1. Memahami tentang Array of Pointer
2. Memahami tentang Pointer to Pointer

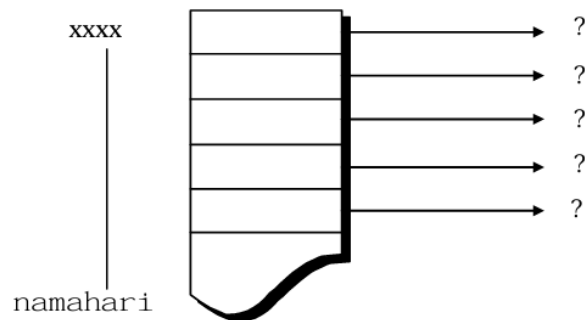
B. DASAR TEORI

Array dari Pointer (Array of Pointer)

- Suatu array bisa digunakan untuk menyimpan sejumlah pointer. Sebagai contoh:

`char *namahari[10];`

merupakan pernyataan untuk mendeklarasikan array pointer. Array namahari terdiri dari 10 elemen berupa pointer yang menunjuk ke data bertipe char. Ilustrasi penempatan dalam memori untuk array of pointer ini dapat dilihat pada gambar 26.1.



Gambar 26.1 Array of Pointer

- Array pointer bisa diinisialisasi sewaktu pendeklarasian. Sebagai contoh:

```
static char *namahari[] =  
{  
    "Senin",  
    "Selasa",  
    "Rabu",  
    "Kamis",  
    "Jumat",  
    "Sabtu",  
    "Minggu"};
```

Pada contoh ini,

namahari[0] berisi alamat yang menunjuk ke string "Senin".

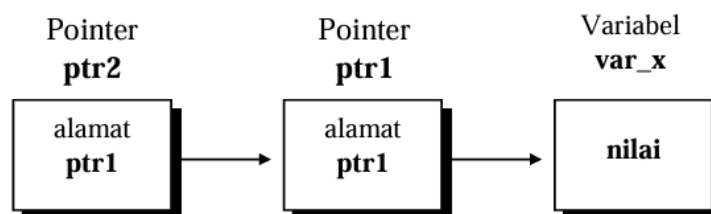
namahari[1] berisi alamat yang menunjuk ke string "Selasa".

namahari[2] berisi alamat yang menunjuk ke string "Rabu".

dan sebagainya.

Pointer menunjuk Pointer (*Pointer to Pointer*)

Suatu pointer bisa saja menunjuk ke pointer lain. Gambar 26.2 memberikan contoh mengenai pointer menunjuk pointer.



Gambar 26.2 Pointer yang menunjuk pointer

- Untuk membentuk rantai pointer seperti pada gambar di atas, pendeklarasian yang diperlukan berupa

```
int var_x;
int *ptr1;
int **ptr2;
```

Perhatikan pada deklarasi di depan:

- var_x adalah variabel bertipe int.
- ptr1 adalah variabel pointer yang menunjuk ke data bertipe int.
- ptr2 adalah variabel pointer yang menunjuk ke pointer int.
- (itulah sebabnya deklarasinya berupa int **ptr2;)
- Agar ptr1 menunjuk ke variabel var_x, perintah yang diperlukan berupa

$$ptr1 = \&var_x;$$
- Sedangkan supaya ptr2 menunjuk ke ptr1, instruksi yang diperlukan adalah $ptr2 = \&ptr1;$

C. PERCOBAAN

Untuk setiap program di bawah ini,

- gambarkan ilustrasi alokasi memori dari setiap baris pernyataan yang diproses
- perkirakan hasil eksekusinya

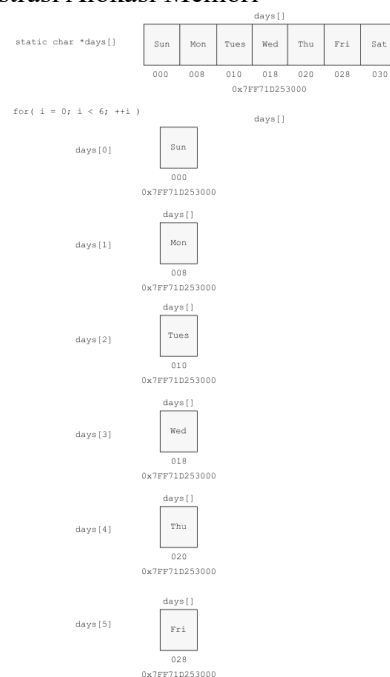
1. Array of Pointer to char

```
main() {
    static char *days[] = {"Sun", "Mon", "Tues", "Wed", "Thu",
                           "Fri", "Sat"};
    int i;

    for( i = 0; i < 6; ++i )
        printf( "%s\n", days[i]);
}
```

Jawab :

a. Ilustrasi Alokasi Memori



- b. Perkiraan hasil eksekusi
Berikut adalah hasil eksekusi pointer

```

PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Praktikum 2\Pointer 3\output> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 3\output'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 3\output> & .\soal 1.exe
Sun
Mon
Tues
Wed
Thu
Fri
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 3\output>

```

Gambar hasil eksekusi pointer

2. Pointer yang menunjuk ke pointer yang lain.

```

main(){
    int a, *b, **c;

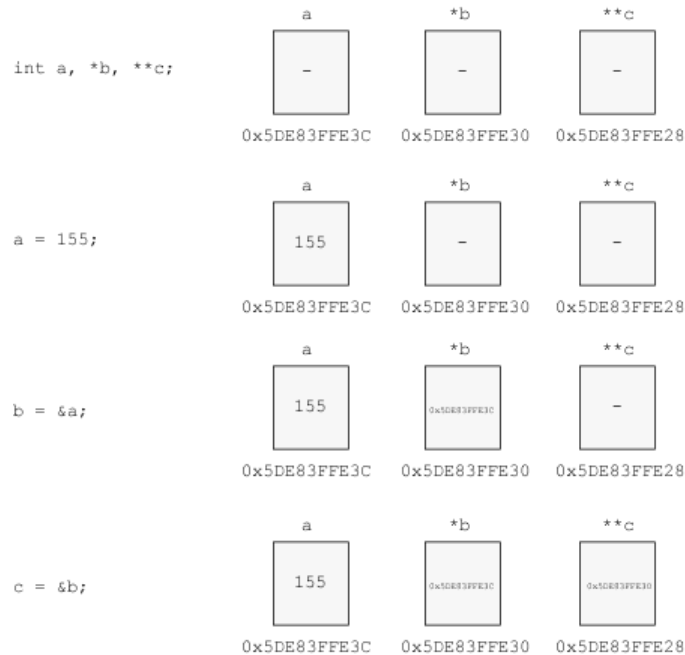
    a = 155;
    b = &a;
    c = &b;

    printf("Nilai a = %d atau %d atau %d\n", a, *b, **c);
    printf("b = %p = alamat a di memori\n", b);
    printf("c = %p = alamat b di memori\n", c);
    printf("alamat c di memori = %p\n", &c);
}

```

Jawab :

- a. Ilustrasi Alokasi Memori



- b. Perkiraan hasil eksekusi
Berikut adalah hasil eksekusi pointer

```

PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Point
er 3> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum
2\Pointer 3\output'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Point
er 3\output> & .\soal 2.exe
Nilai a = 155 atau 155 atau 155
b = 0000005DE83FFE3C = alamat a di memori
c = 0000005DE83FFE30 = alamat b di memori
alamat c di memori = 0000005DE83FFE28
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Point
er 3\output>

```

Gambar hasil eksekusi pointer

3. Pointer yang menunjuk ke pointer yang lain.

```

main(){
    int var_x = 273;
    int *ptr1;
    int **ptr2;

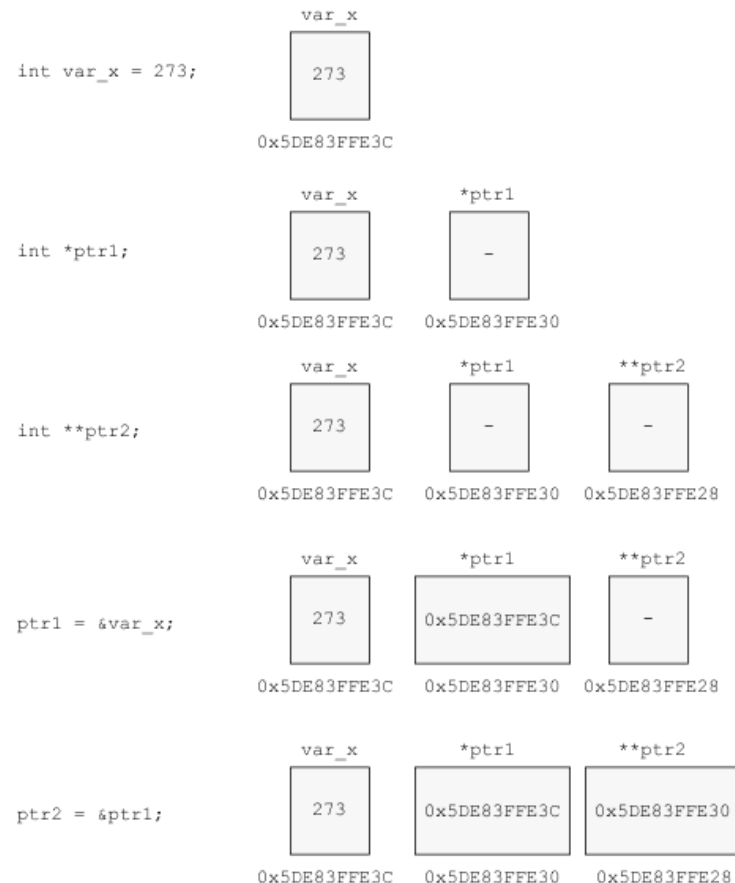
    ptr1 = &var_x;
    ptr2 = &ptr1;

    printf("Nilai var_x = *ptr1 = %d\n", *ptr1);
    printf("Nilai var_x = **ptr2 = %d\n\n", **ptr2);
    printf("ptr1 = &var_x = %p\n", ptr1);
    printf("ptr2 = &ptr1 = %p\n", ptr2);
    printf("      &ptr2 = %p\n", &ptr2);
}

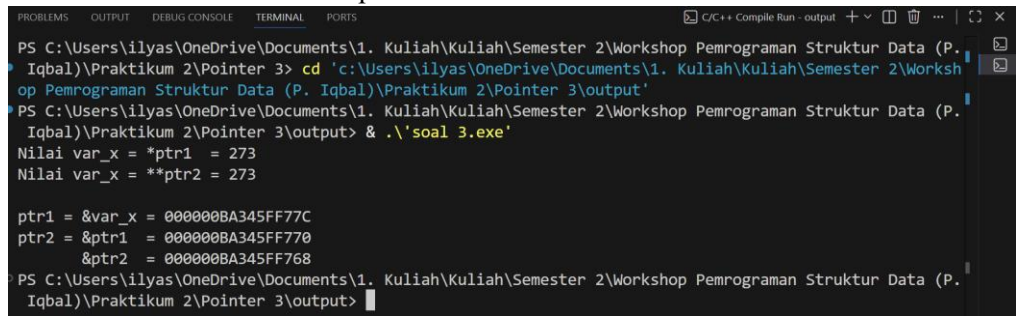
```

Jawab :

- a. Ilustrasi Alokasi Memori



- Berikut adalah hasil eksekusi pointer



Gambar hasil eksekusi pointer

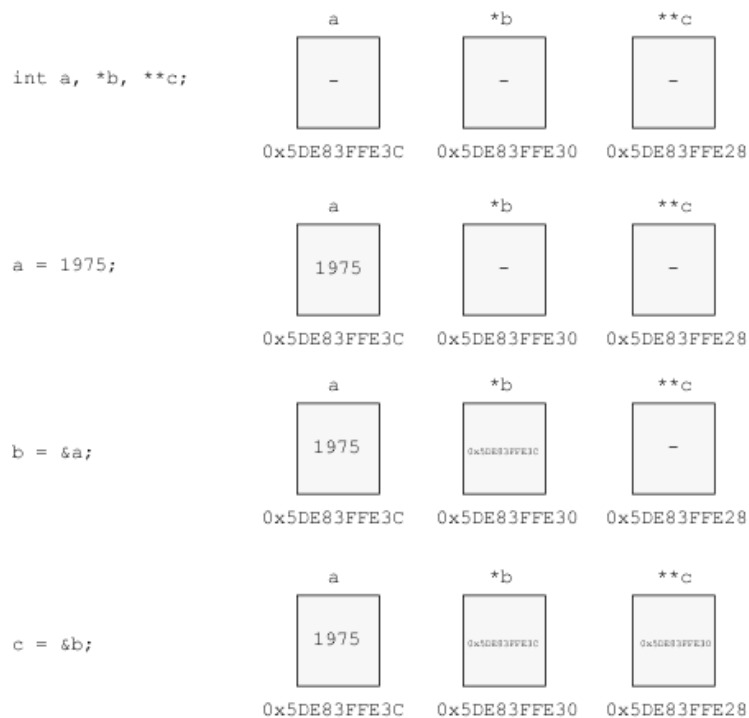
```
4. #include <stdio.h>
   main(){
       int a, *b, **c;

       a = 1975;
       b = &a;
       c = &b;

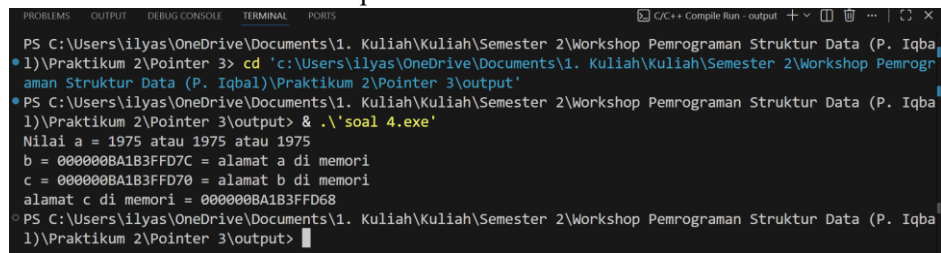
       printf("Nilai a = %d atau %d atau %d\n", a, *b, **c);
       printf("b = %p = alamat a di memori\n", b);
       printf("c = %p = alamat b di memori\n", c);
       printf("alamat c di memori = %p\n", &c);
   }
```

Jawab :

- a. Ilustrasi alokasi memori



- b. Perkiraan hasil eksekusi
Berikut adalah hasil eksekusi pointer



```
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 3> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 3\output'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 3\output> & .\soal 4.exe
Nilai a = 1975 atau 1975 atau 1975
b = 000000BA1B3FFD7C = alamat a di memori
c = 000000BA1B3FFD70 = alamat b di memori
alamat c di memori = 000000BA1B3FFD68
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 3\output>
```

Gambar hasil eksekusi pointer

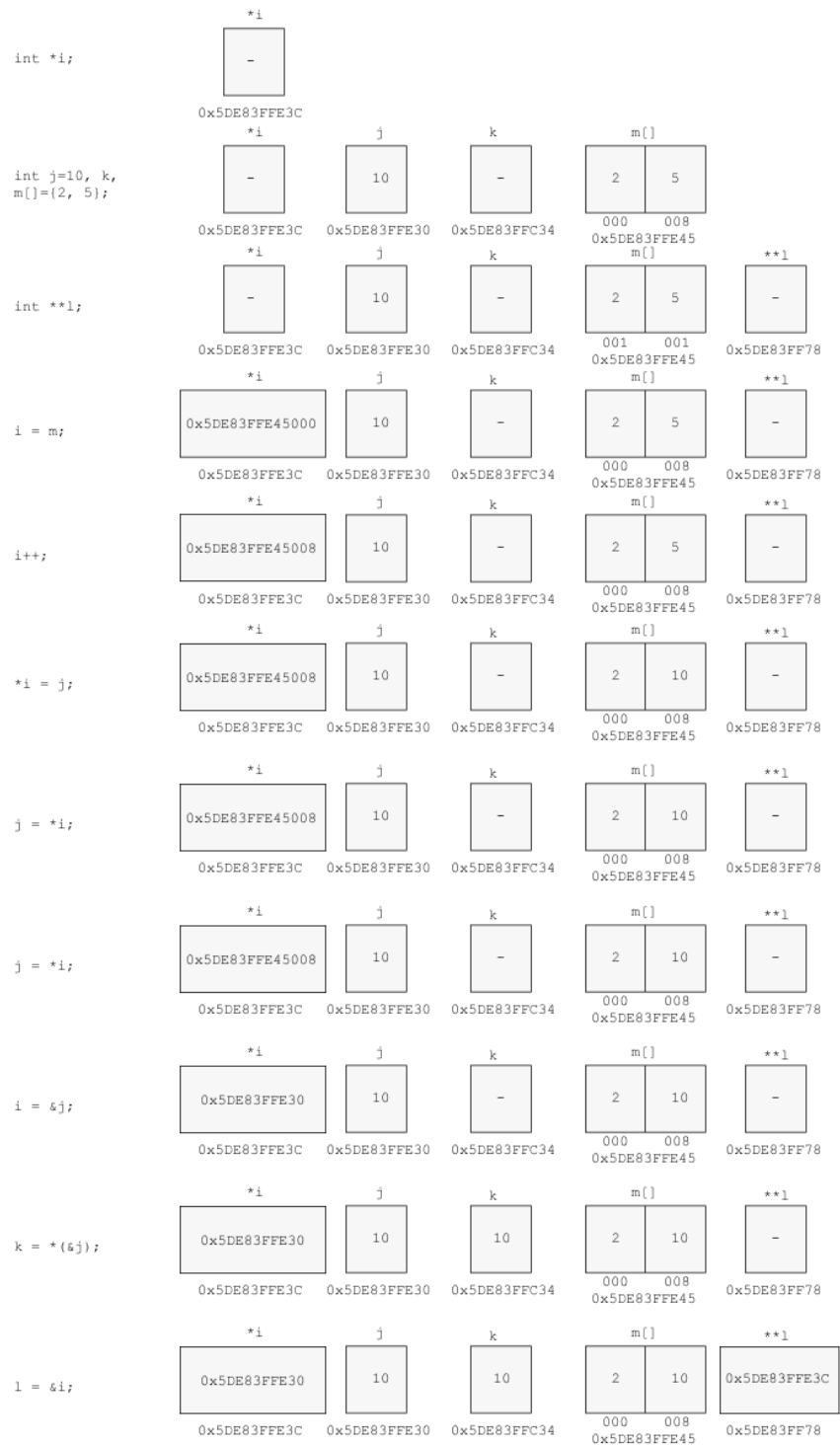
5. Untuk potongan program di bawah ini, gambarkan ilustrasi alokasi memori dari setiap baris pernyataan yang diproses

```
int *i;
int j=10, k, m[]={2, 5};
int **l;
```

```
i = m;
i++;
*i = j;
j = *i;
i = &j;
k = *(&j);
l = &i;
```

Jawab :

- a. Ilustrasi Alokasi Memori



Ilustrasi alokasi memori

6. Tentukan setiap statemen di bawah ini benar atau salah. Jika salah sertakan alasannya.
Deklarasi :

```
int a[5] = {2,4,8,1,23};
int c = 5;
int *ptr1 = &c;
```

```
int *ptr2 = a;
```

Statement	Benar (beri tanda X)	Salah (beri tanda X)	Alasan
<code>a = c;</code>		X	a adalah nama array (konstanta pointer), nilainya (alamat memori) tidak bisa diubah dengan variabel biasa.
<code>*c = 6;</code>		X	c adalah variabel int, bukan pointer. Operator dereferensi * hanya bisa digunakan pada pointer.
<code>a[2] = c;</code>	X		
<code>*ptr2 = c;</code>	X		
<code>&ptr1 = c;</code>		X	&ptr1 menghasilkan alamat dari pointer. Alamat memori adalah <i>read-only</i> (R-value) dan tidak bisa diberi nilai baru.
<code>*(ptr2 + 1) = *(a + 3);</code>	X		
<code>c = *(ptr2 + 1);</code>	X		
<code>c = &ptr1;</code>		X	Ketidakcocokan tipe data. c adalah int, sedangkan &ptr1 adalah alamat dari pointer (int**).
<code>c = a[3] + 2;</code>	X		
<code>*(ptr2 + 2) = *ptr1;</code>	X		

D. KESIMPULAN

Berdasarkan hasil analisis terhadap berbagai skenario percobaan, dapat disimpulkan bahwa penggunaan pointer menawarkan efisiensi dan fleksibilitas yang lebih tinggi dibandingkan akses array statis karena memungkinkan manipulasi data langsung melalui alamat memori. Hal ini terbukti dari kemampuan pointer dalam melakukan penukaran string secara instan melalui referensi alamat, serta kemampuannya menelusuri atau mengkloning array dengan operasi aritmatika yang lebih cepat daripada metode penyalinan konten tradisional. Dengan demikian, penguasaan terhadap logika dereferensi, *pointer arithmetic*, dan pemahaman hierarki alamat memori menjadi fondasi krusial dalam menciptakan program yang optimal, sistematis, dan memiliki kontrol sumber daya yang presisi pada bahasa C.

LAPORAN PRAKTIKUM 1

Pointer 4



Disusun Oleh:

Ilyas Rahman Salim (5325600063)

Program Studi D4 Teknologi Rekayasa Multimedia

Departemen Teknologi Multimedia Kreatif

Politeknik Elektronika Negeri Surabaya

2026

A. TUJUAN PEMBELAJARAN

1. Memahami tentang Pointer dalam Fungsi
2. Memahami tentang Pointer sebagai Parameter Fungsi
3. Memahami tentang Pointer sebagai Keluaran Fungsi

B. DASAR TEORI

Pointer dalam Fungsi

Pointer dan kaitannya dengan fungsi yang akan dibahas berikut meliputi :

- Pointer sebagai parameter fungsi
- Pointer sebagai keluaran fungsi

Pointer Sebagai Parameter Fungsi

- Penerapan pointer sebagai parameter yaitu jika diinginkan agar nilai suatu variabel internal dapat diubah oleh fungsi yang dipanggil.
- Sebagai contoh dapat dilihat pada fungsi berikut.

```
void naikkan_nilai (int *x, int *y){
    *x = *x + 2;
    *y = *y + 2;
}
```
- Fungsi di atas dimaksudkan agar kalau dipanggil, variabel yang berkenaan dengan parameter aktual dapat diubah nilainya, masing-masing dinaikkan sebesar 2. Contoh pemanggilan :

```
naikkan_nilai(&a, &b);
```
- Perhatikan, dalam hal ini variabel a dan b harus ditulis diawali operator alamat (&) yang berarti menyatakan alamat variabel, sebab parameter fungsi dalam pendefinisian berupa pointer.

Pointer Sebagai Keluaran Fungsi (return value)

- Suatu fungsi dapat dibuat agar keluarannya berupa pointer. Misalnya, suatu fungsi menghasilkan keluaran berupa pointer yang menunjuk ke string nama_bulan, seperti pada contoh berikut.

```
char *nama_bulan(int n) {
    static char *bulan[]=
        {"Kode bulan salah", "Januari", "Februari", "Maret",
         "April", "Mei", "Juni", "Juli", "Agustus",
         "September", "Oktober", "Nopember", "Desember"};
    return ( (n<1 || n>12) ? bulan[0] : bulan[n] );
}
```

- Pada definisi fungsi di atas,

```
char *nama_bulan()
```

menyatakan bahwa keluaran fungsi nama_bulan() berupa pointer yang menunjuk ke obyek char (atau string).
- Dalam fungsi nama_bulan(), mula-mula array bernama bulan dideklarasikan dan sekaligus diinisialisasi agar menunjuk sejumlah string yang menyatakan nama bulan. Di bagian akhir fungsi, pernyataan

```
return ( (n<1 || n>12) ? bulan[0] : bulan[n] );
```

- menyatakan bahwa hasil fungsi berupa pointer yang menunjuk ke
- string “Kode bulan salah” (bulan[0]) jika masukan fungsi n12
 - bulan[n] untuk n yang terletak antara 1 sampai dengan 12.

C. PERCOBAAN

Untuk setiap program di bawah ini,

- gambarkan ilustrasi alokasi memori dari setiap baris pernyataan yang diproses
- perkirakan hasil eksekusinya

1. Fungsi dengan argumen berupa pointer.

```
#include <stdio.h>
void naikkan_nilai(int *x, int *y);

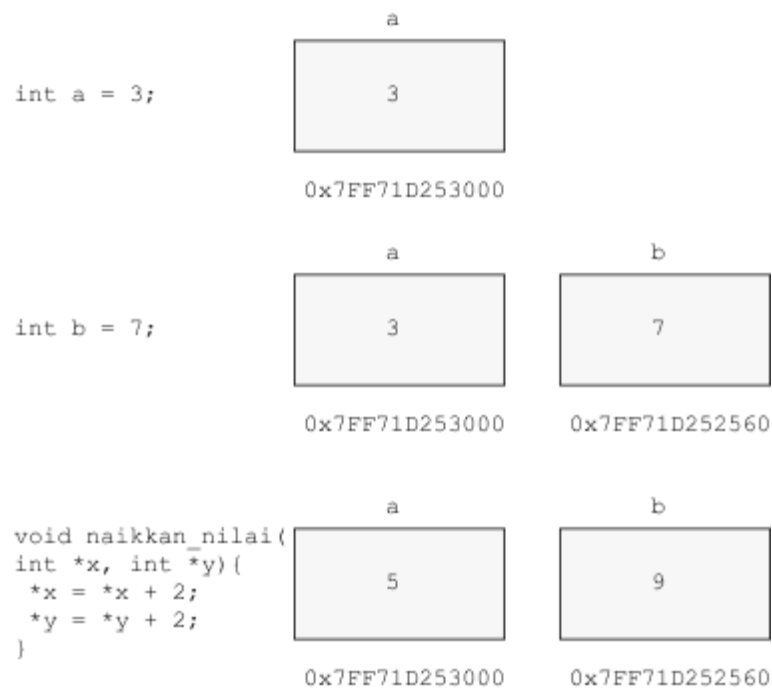
main() {
    int a = 3;
    int b = 7;

    printf("SEMULA : a = %d b = %d\n", a, b);
    naikkan_nilai(&a, &b);
    printf("KINI    : a = %d b = %d\n", a, b);
}

void naikkan_nilai(int *x, int *y){
    *x = *x + 2;
    *y = *y + 2;
}
```

Jawab :

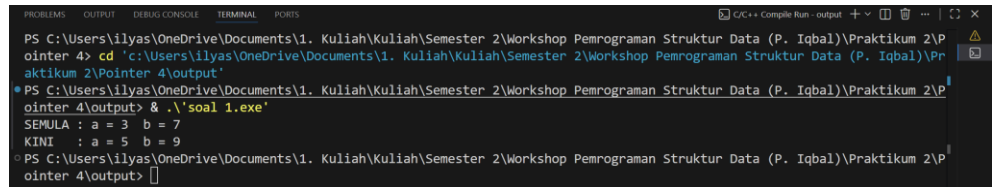
- a. Ilustrasi alokasi memori



Ilustrasi alokasi memori

- b. Perkiraan hasil eksekusi
Berikut adalah hasil eksekusi pointer

Gambar hasil eksekusi pointer



```
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4\output'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4\output> & .\soal 1.exe
SEMULA : a = 3  b = 7
KINI    : a = 5  b = 9
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4\output>
```

2. Fungsi dengan keluaran berupa pointer yang menunjuk string.

```
#include <stdio.h>
char *nama_bulan(int n);

main(){
    int bl;

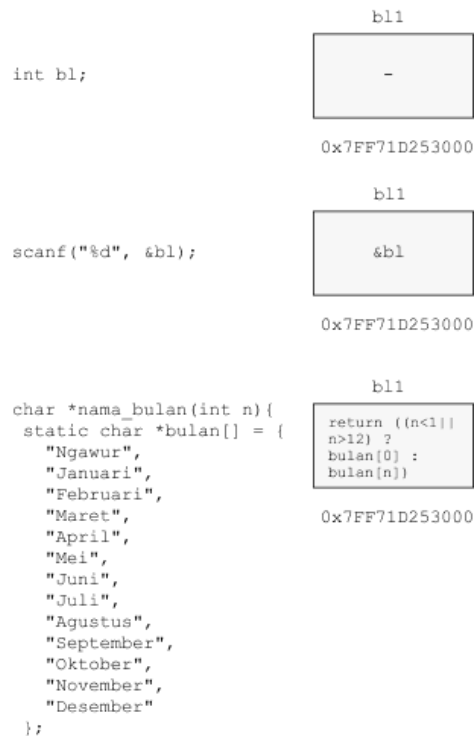
    printf("Bulan 1..12 : ");
    scanf("%d", &bl);
    printf("Bulan ke-%d adalah %s\n", bl, nama_bulan(bl));
}

char *nama_bulan(int n){
    static char *bulan[] = {
        "Ngawur",
        "Januari",
        "Februari",
        "Maret",
        "April",
        "Mei",
        "Juni",
        "Juli",
        "Agustus",
        "September",
        "Oktober",
        "November",
        "Desember"
    };

    return ((n<1||n>12) ? bulan[0] : bulan[n]);
}
```

Jawab :

- a. Ilustrasi alokasi memori



Ilustrasi alokasi memori

b. Perkiraan hasil eksekusi

Berikut adalah hasil eksekusi pointer

```

PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4\output> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4\output'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4\output> & .\soal 2.exe
Bulan 1..12 : 6
Bulan ke-6 adalah Juni
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4\output>

```

Gambar hasil eksekusi pointer

Berdasarkan gambar tersebut ketika diinputkan angka 6, maka akan muncul Bulan ke-6 adalah Juni.

3. Berikan ilustrasi dan jelaskan apa yang dilakukan oleh program di bawah ini dan tampilkan hasil eksekusinya..

```

#include <stdio.h>

char strA[80] = "A string to be used for demonstration";
char strB[80];

main(){
    char *pA, *pB;

    puts(strA);
    pA = strA;
    puts(pA);
    pB = strB;
    putchar('\n');
}

```

```

while(*pA != '\0')
    *pB++ = *pA++;

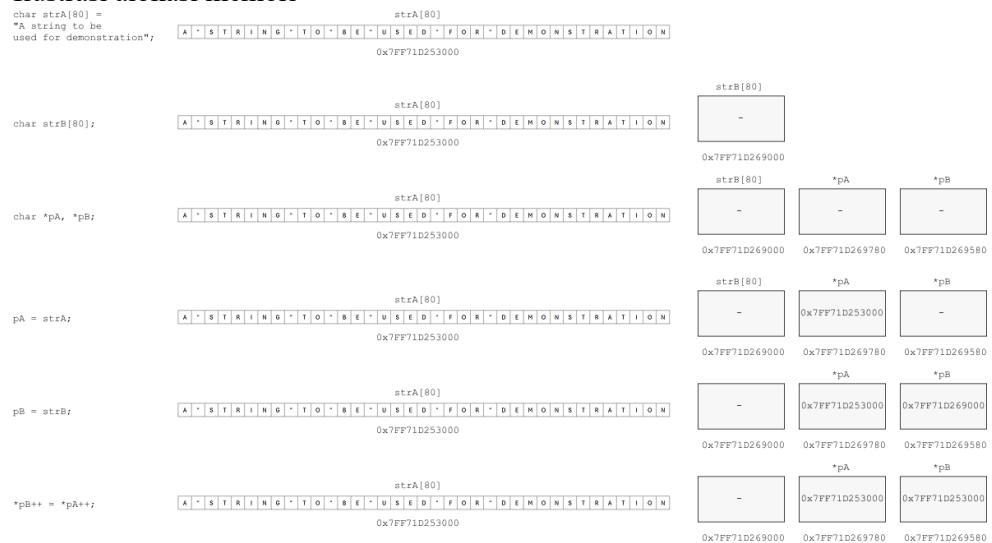
*pB = '\0';
puts(strB);
}

```

Proses apakah yang sebenarnya dilakukan pada program tersebut ?

Jawab :

a. Ilustrasi alokasi memori



Ilustrasi alokasi memori

b. Perkiraan hasil eksekusi

Berikut adalah hasil eksekusi pointer

```

PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4\output'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4\output> & .\soal 3.exe
A string to be used for demonstration
A string to be used for demonstration
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4\output>

```

Gambar hasil eksekusi pointer

c. Proses apakah yang sebenarnya dilakukan pada program tersebut ?

Secara teknis, program ini melakukan transfer data antar alamat memori. Alih-alih memindahkan seluruh blok teks sekaligus, program memanfaatkan sifat pointer yang bisa bergerak selangkah demi selangkah (increment) untuk memetakan isi dari satu array ke array lainnya.

4. Berikan ilustrasi dan jelaskan apa yang dilakukan oleh program di bawah ini

```

#include <stdio.h>

char *my_strcpy(char *, char *);

main(){

```

```
char strA[80]="A string to be used for demonstration";
char strB[80];
```

```
my_strcpy(strB, strA);
puts(strB);
}
```

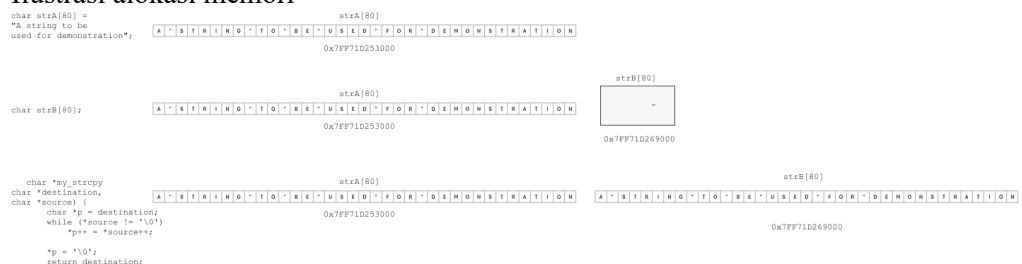
23

```
char *my_strcpy(char *destination, char *source) {
    char *p = destination;
    while (*source != '\0')
        *p++ = *source++;
```

```
    *p = '\0';
    return destination;
}
```

Jawab :

a. Ilustrasi alokasi memori



Ilustrasi alokasi memori

b. Perkiraan hasil eksekusi

Berikut adalah hasil eksekusi pointer

```
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Point
er 4> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum
2\Pointer 4\output'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Point
er 4\output> & .\soal 4.exe
A string to be used for demonstration
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Point
er 4\output>
```

Gambar hasil eksekusi pointer

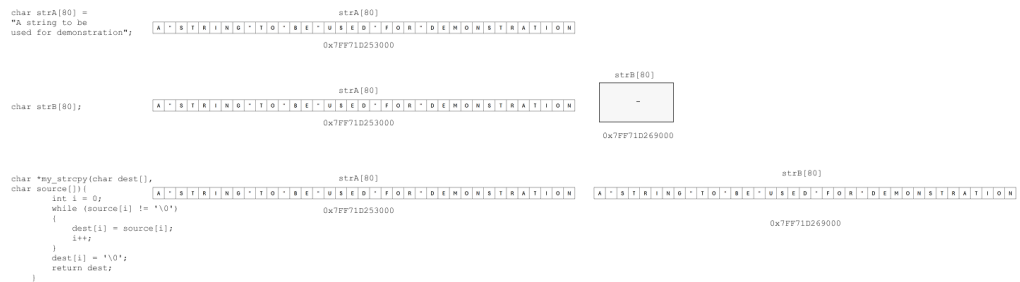
5. Bandingkan fungsi `my_strcpy()` di atas dengan fungsi `my_strcpy()` di bawah ini. Berikan penjelasan terhadap perbedaan proses dari kedua fungsi tersebutint *i;

```
char *my_strcpy(char dest[], char source[]){
    int i = 0;
    while (source[i] != '\0')
    {
        dest[i] = source[i];
        i++;
    }
    dest[i] = '\0';
    return dest;
```

```
}
```

Jawab :

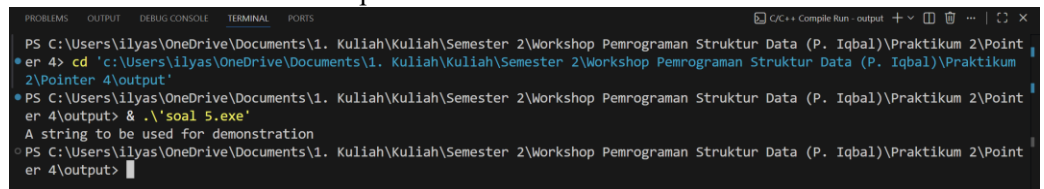
a. Ilustrasi alokasi memori



Ilustrasi alokasi memori

b. Perkiraan hasil eksekusi

Berikut adalah hasil eksekusi pointer



Gambar hasil eksekusi pointer

c. Perbedaan Proses

- Fungsi Pertama (Pointer): Menggunakan "jari" yang terus bergeser (*p++). Program langsung memanipulasi alamat memori untuk berpindah dari satu karakter ke karakter berikutnya. Ini dianggap lebih "dekat" dengan cara mesin bekerja.
- Fungsi Kedua (Indexing): Menggunakan variabel penghitung (i) sebagai koordinat. Program harus menghitung ulang posisi setiap kali ingin mengambil karakter: "Ambil karakter di posisi awal ditambah i langkah".

- Buatlah sebuah program dengan mendefinisikan sebuah fungsi rotasi() yang menerima tiga parameter berupa variabel a, b, dan c. Fungsi ini melakukan rotasi sehingga nilai a berpindah ke b, b ke c dan nilai c ke a sekembalinya ke fungsi main().

Jawab :

a. Source Code

```

#include <stdio.h>

void rotasi(int *a, int *b, int *c) {
    int temp;

    temp = *c;
    *c = *b;
    *b = *a;
    *a = temp;
}

int main() {
    int a, b, c;

```

```

printf("Masukkan nilai a: "); scanf("%d", &a);
printf("Masukkan nilai b: "); scanf("%d", &b);
printf("Masukkan nilai c: "); scanf("%d", &c);

printf("\nNilai sebelum rotasi:\n");
printf("a = %d, b = %d, c = %d\n", a, b, c);

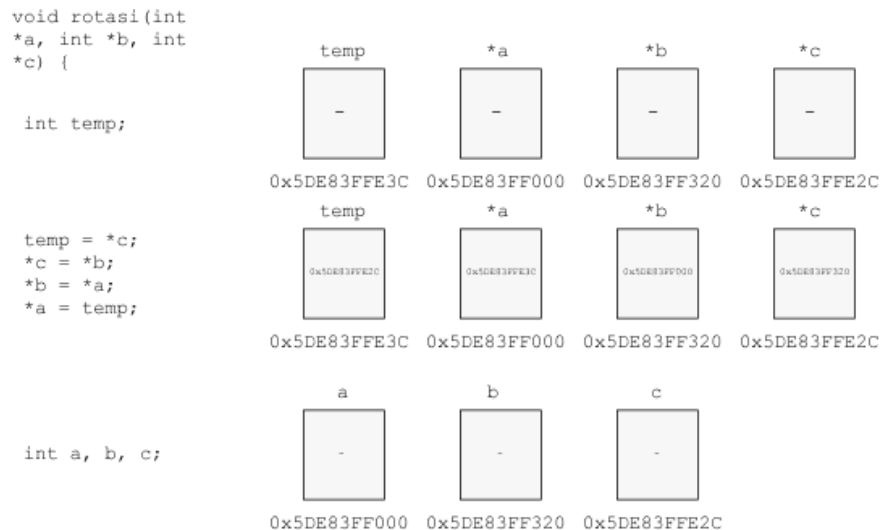
rotasi(&a, &b, &c);

printf("\nNilai setelah rotasi (a->b, b->c, c->a):\n");
printf("a = %d, b = %d, c = %d\n", a, b, c);

return 0;
}

```

b. Ilustrasi alokasi memori



Ilustrasi alokasi memori

c. Perkiraan hasil eksekusi

Berikut adalah hasil eksekusi pointer

```

PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4\output> cd 'c:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4\output'
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4\output> & .\soal 6.exe
Masukkan nilai a: 10
Masukkan nilai b: 20
Masukkan nilai c: 30

Nilai sebelum rotasi:
a = 10, b = 20, c = 30

Nilai setelah rotasi (a->b, b->c, c->a):
a = 30, b = 10, c = 20
PS C:\Users\ilyas\OneDrive\Documents\1. Kuliah\Kuliah\Semester 2\Workshop Pemrograman Struktur Data (P. Iqbal)\Praktikum 2\Pointer 4\output>

```

Gambar hasil eksekusi pointer

D. KESIMPULAN

Berdasarkan hasil praktikum terhadap serangkaian percobaan yang telah dilakukan, dapat disimpulkan bahwa implementasi fungsi **rotasi()** mempertegas peran **pointer** sebagai instrumen vital dalam melakukan perubahan data secara permanen lintas lingkup fungsi

(*function scope*). Melalui mekanisme **pass-by-reference**, program mampu melakukan pertukaran nilai antar tiga variabel secara simultan dengan memanipulasi alamat memori aslinya menggunakan operator *dereference* (*), sehingga sinkronisasi data antara fungsi `main()` dan fungsi `rotasi()` dapat tercapai dengan akurat. Penggunaan variabel bantu temp dalam algoritma ini menunjukkan pentingnya manajemen penyimpanan sementara saat melakukan rotasi nilai, sementara penggunaan simbol ampersand (&) saat pemanggilan fungsi membuktikan bahwa yang dikirimkan adalah lokasi memori, bukan sekadar nilai salinan. Secara keseluruhan, praktik ini memberikan pemahaman komprehensif bahwa penguasaan pointer tidak hanya mengoptimalkan efisiensi eksekusi program, tetapi juga menjadi fondasi utama dalam merancang logika pemrosesan data yang dinamis dan sistematis pada bahasa pemrograman tingkat menengah. sangat krusial untuk mengelola alur data antar fungsi secara efisien dan sistematis.